YEI 3-Space C API

Generated by Doxygen 1.8.0

Wed Jun 13 2012 15:38:06

Contents

1 Data Structure Index **1**

1.1 Data Structures 1

2 File Index **3**

2.1 File List 3

3 Data Structure Documentation **5**

3.1 AxisDirections Struct Reference 5

3.1.1 Detailed Description 5

3.2 BootloaderData Struct Reference 5

3.2.1 Detailed Description 6

3.3 ButtonState Struct Reference 6

3.3.1 Detailed Description 6

3.4 ComInfo Struct Reference 6

3.4.1 Detailed Description 7

3.5 ComPort Struct Reference 7

3.5.1 Detailed Description 7

3.6 SensorData Struct Reference 7

3.6.1 Detailed Description 8

3.7 TSS_AxisAngle Struct Reference 8

3.7.1 Detailed Description 9

3.8 TSS_Clock Struct Reference 9

3.8.1 Detailed Description 9

3.8.2 Field Documentation 9

3.8.2.1 day 9

3.8.2.2 hour 9

3.8.2.3 minute 9

3.8.2.4 month 10

3.8.2.5 second 10

3.8.2.6 year 10

3.9 TSS_Color Struct Reference 10

3.9.1 Detailed Description 10

3.10 TSS_Euler Struct Reference . . . 10
3.10.1 Detailed Description . . . 11
3.11 TSS_Matrix Struct Reference . . . 11
3.11.1 Detailed Description . . . 11
3.12 TSS_Quaternion Struct Reference . . . 12
3.12.1 Detailed Description . . . 12
3.13 TSS_Vector Struct Reference . . . 12
3.13.1 Detailed Description . . . 12

4 File Documentation **13**

4.1 include/yei_threespace_api.h File Reference . . . 13
4.1.1 Detailed Description . . . 30
4.1.2 Typedef Documentation . . . 30
4.1.2.1 Axis_Order . . . 30
4.1.2.2 ButtonState . . . 30
4.1.2.3 SensorData . . . 30
4.1.2.4 TSS_AxisAngle . . . 31
4.1.2.5 TSS_Euler . . . 31
4.1.2.6 TSS_Matrix . . . 31
4.1.3 Enumeration Type Documentation . . . 31
4.1.3.1 Axis_Order . . . 31
4.1.3.2 TSS_Error . . . 32
4.1.4 Function Documentation . . . 32
4.1.4.1 addSensorToDongle . . . 32
4.1.4.2 asynchGetAccelerometerNormalized . . . 33
4.1.4.3 asynchGetAccelerometerRaw . . . 33
4.1.4.4 asynchGetAccelerometerUnfiltered . . . 34
4.1.4.5 asynchGetAllSensorsNormalized . . . 34
4.1.4.6 asynchGetAllSensorsRaw . . . 35
4.1.4.7 asynchGetButtonState . . . 35
4.1.4.8 asynchGetCompassNormalized . . . 36
4.1.4.9 asynchGetCompassRaw . . . 36
4.1.4.10 asynchGetCompassUnfiltered . . . 37
4.1.4.11 asynchGetFiltNorthEarthVecSensFrame . . . 37
4.1.4.12 asynchGetFiltOrientAxisAngle . . . 38
4.1.4.13 asynchGetFiltOrientEuler . . . 38
4.1.4.14 asynchGetFiltOrientFwdDwn . . . 39
4.1.4.15 asynchGetFiltOrientMat . . . 39
4.1.4.16 asynchGetFiltOrientQuat . . . 40
4.1.4.17 asynchGetFiltTaredFwdDwnVecSensFrame . . . 40

4.1.4.18 asynchGetFiltTaredOrientAxisAngle . . . 41
4.1.4.19 asynchGetFiltTaredOrientEuler . . . 41
4.1.4.20 asynchGetFiltTaredOrientFwdDwn . . . 42
4.1.4.21 asynchGetFiltTaredOrientMatrix . . . 42
4.1.4.22 asynchGetFiltTaredOrientQuat . . . 43
4.1.4.23 asynchGetGyroNormalized . . . 43
4.1.4.24 asynchGetGyroRaw . . . 44
4.1.4.25 calibrateGyro . . . 44
4.1.4.26 closeTSDevice . . . 44
4.1.4.27 commitSettings . . . 45
4.1.4.28 commitWirelessSettings . . . 45
4.1.4.29 createTSDevice . . . 45
4.1.4.30 createTSDeviceStr . . . 45
4.1.4.31 disableAxis . . . 46
4.1.4.32 disableButton . . . 46
4.1.4.33 disableMassStorageMode . . . 46
4.1.4.34 disableWatchdogTimer . . . 47
4.1.4.35 enableMassStorageMode . . . 47
4.1.4.36 enableWatchdogTimer . . . 47
4.1.4.37 enterFirmwareUpdateMode . . . 47
4.1.4.38 finishWrite . . . 48
4.1.4.39 formatInitSDCard . . . 48
4.1.4.40 getAccelerometerCalibrationParam . . . 48
4.1.4.41 getAccelerometerCalibrationParamf9f3 . . . 48
4.1.4.42 getAccelerometerEnabled . . . 49
4.1.4.43 getAccelerometerNormalized . . . 49
4.1.4.44 getAccelerometerNormalizedf3 . . . 49
4.1.4.45 getAccelerometerRange . . . 50
4.1.4.46 getAccelerometerRaw . . . 50
4.1.4.47 getAccelerometerRawf3 . . . 50
4.1.4.48 getAccelerometerUnfiltered . . . 51
4.1.4.49 getAccelerometerUnfilteredf3 . . . 51
4.1.4.50 getActualUpdateRate . . . 51
4.1.4.51 getAllChannelNoise . . . 52
4.1.4.52 getAllSensorsNormalized . . . 52
4.1.4.53 getAllSensorsNormalizedf9 . . . 52
4.1.4.54 getAllSensorsRaw . . . 52
4.1.4.55 getAllSensorsRawf9 . . . 53
4.1.4.56 getAsynchDataFloat . . . 53
4.1.4.57 getAsynchDataUChar . . . 53

4.1.4.58 getAsynchDataUInt . . . 54
4.1.4.59 getAsynchDataUShort . . . 54
4.1.4.60 getAvailableComPorts . . . 55
4.1.4.61 getAvgPercent . . . 55
4.1.4.62 getAxisDirections . . . 55
4.1.4.63 getBatteryLife . . . 56
4.1.4.64 getBatteryStatus . . . 56
4.1.4.65 getBatteryVoltage . . . 56
4.1.4.66 getButtonGyroDisableLength . . . 57
4.1.4.67 getButtonState . . . 57
4.1.4.68 getChannel . . . 57
4.1.4.69 getClock . . . 57
4.1.4.70 getClockc6 . . . 58
4.1.4.71 getClockSpeed . . . 58
4.1.4.72 getCompassCalibrationParam . . . 58
4.1.4.73 getCompassCalibrationParamf9f3 . . . 59
4.1.4.74 getCompassEnabled . . . 59
4.1.4.75 getCompassNormalized . . . 59
4.1.4.76 getCompassNormalizedf3 . . . 60
4.1.4.77 getCompassRange . . . 60
4.1.4.78 getCompassRaw . . . 60
4.1.4.79 getCompassRawf3 . . . 61
4.1.4.80 getCompassUnfiltered . . . 61
4.1.4.81 getCompassUnfilteredf3 . . . 61
4.1.4.82 getComPortInfo . . . 61
4.1.4.83 getConfidence . . . 62
4.1.4.84 getDesiredUpdateRate . . . 62
4.1.4.85 getDeviceType . . . 62
4.1.4.86 getFilterMode . . . 63
4.1.4.87 getFiltGyroRate . . . 63
4.1.4.88 getFiltGyroRatef4 . . . 63
4.1.4.89 getFiltNorthEarthVecSensFrame . . . 64
4.1.4.90 getFiltNorthEarthVecSensFramef3f3 . . . 64
4.1.4.91 getFiltOrientAxisAngle . . . 64
4.1.4.92 getFiltOrientAxisAnglef4 . . . 65
4.1.4.93 getFiltOrientEuler . . . 65
4.1.4.94 getFiltOrientEulerf3 . . . 65
4.1.4.95 getFiltOrientFwdDwn . . . 66
4.1.4.96 getFiltOrientFwdDwnf3f3 . . . 66
4.1.4.97 getFiltOrientMat . . . 66

4.1.4.98 getFiltOrientMatf9 . . . 67
4.1.4.99 getFiltOrientQuat . . . 67
4.1.4.100 getFiltOrientQuatf4 . . . 67
4.1.4.101 getFiltTaredFwdDwnVecSensFrame . . . 67
4.1.4.102 getFiltTaredFwdDwnVecSensFramef3f3 . . . 68
4.1.4.103 getFiltTaredOrientAxisAngle . . . 68
4.1.4.104 getFiltTaredOrientAxisAnglef4 . . . 68
4.1.4.105 getFiltTaredOrientEuler . . . 69
4.1.4.106 getFiltTaredOrientEulerf3 . . . 69
4.1.4.107 getFiltTaredOrientFwdDwn . . . 69
4.1.4.108 getFiltTaredOrientFwdDwnf3f3 . . . 70
4.1.4.109 getFiltTaredOrientMat . . . 70
4.1.4.110 getFiltTaredOrientMatf9 . . . 70
4.1.4.111 getFiltTaredOrientQuat . . . 70
4.1.4.112 getFiltTaredOrientQuatf4 . . . 71
4.1.4.113 getFirstAvailableTSSComPort . . . 71
4.1.4.114 getFirstAvailableUnknownComPort . . . 71
4.1.4.115 getFriendlyName . . . 71
4.1.4.116 getGyroCalibrationParam . . . 72
4.1.4.117 getGyroCalibrationParamf9f3 . . . 72
4.1.4.118 getGyroEnabled . . . 72
4.1.4.119 getGyroNormalized . . . 73
4.1.4.120 getGyroNormalizedf3 . . . 73
4.1.4.121 getGyroRange . . . 73
4.1.4.122 getGyroRaw . . . 74
4.1.4.123 getGyroRawf3 . . . 74
4.1.4.124 getHardwareVersion . . . 74
4.1.4.125 getHIDUpdateRate . . . 74
4.1.4.126 getInformation . . . 75
4.1.4.127 getInterruptStatus . . . 75
4.1.4.128 getInterruptType . . . 75
4.1.4.129 getJoystickEnabled . . . 76
4.1.4.130 getJoystickLogicalID . . . 76
4.1.4.131 getJoystickMousePresent . . . 77
4.1.4.132 getKalmanMat . . . 77
4.1.4.133 getKalmanMatf9 . . . 77
4.1.4.134 getLEDColor . . . 78
4.1.4.135 getLEDColorf3 . . . 78
4.1.4.136 getLEDMode . . . 78
4.1.4.137 getLookupTblVertVal . . . 79

4.1.4.138 getLookupTblVertValf3 . . . 79
4.1.4.139 getMouseEnabled . . . 79
4.1.4.140 getMouseLogicalID . . . 80
4.1.4.141 getMouseRelative . . . 80
4.1.4.142 getMultiRefChkVecAccelerometer . . . 80
4.1.4.143 getMultiRefChkVecAccelerometerf3 . . . 81
4.1.4.144 getMultiRefChkVecCompass . . . 81
4.1.4.145 getMultiRefChkVecCompassf3 . . . 81
4.1.4.146 getMultiRefResolution . . . 82
4.1.4.147 getMultiRefVecAccelerometer . . . 82
4.1.4.148 getMultiRefVecAccelerometerf3 . . . 82
4.1.4.149 getMultiRefVecCompass . . . 83
4.1.4.150 getMultiRefVecCompassf3 . . . 83
4.1.4.151 getMultiRefWeightPwr . . . 83
4.1.4.152 getNextAvailableTSSComPort . . . 84
4.1.4.153 getNextAvailableUnknownComPort . . . 84
4.1.4.154 getNumMultiRefCells . . . 84
4.1.4.155 getOversampleRate . . . 84
4.1.4.156 getPanID . . . 85
4.1.4.157 getReceptionStrength . . . 85
4.1.4.158 getRefVecAccelerometer . . . 85
4.1.4.159 getRefVecAccelerometerf3 . . . 86
4.1.4.160 getRefVecCompass . . . 86
4.1.4.161 getRefVecCompassf3 . . . 86
4.1.4.162 getRefVecMode . . . 86
4.1.4.163 getRhoDataAccelerometer . . . 87
4.1.4.164 getRhoDataCompass . . . 87
4.1.4.165 getRunningAvgMode . . . 88
4.1.4.166 getSensorFromDongle . . . 88
4.1.4.167 getSerialNumber . . . 88
4.1.4.168 getSerialNumberHex . . . 89
4.1.4.169 getSoftwareVersion . . . 89
4.1.4.170 getTareOrientMat . . . 89
4.1.4.171 getTareOrientMatf9 . . . 89
4.1.4.172 getTareOrientQuat . . . 90
4.1.4.173 getTareOrientQuatf4 . . . 90
4.1.4.174 getTemperatureCelsius . . . 90
4.1.4.175 getTemperatureFahrenheit . . . 90
4.1.4.176 getUARTRate . . . 91
4.1.4.177 getUSBMode . . . 91

4.1.4.178 getWirelessAddress . . . 91
4.1.4.179 getWirelessHWID . . . 92
4.1.4.180 getWirelessRetries . . . 92
4.1.4.181 getWirelessSlotsOpen . . . 92
4.1.4.182 isConnected . . . 93
4.1.4.183 isFirmwareValid . . . 93
4.1.4.184 resetKalmanFilter . . . 93
4.1.4.185 restoreFactorySettings . . . 94
4.1.4.186 setAccelerometerCalibrationParam . . . 94
4.1.4.187 setAccelerometerCalibrationParamf9f3 . . . 94
4.1.4.188 setAccelerometerEnabled . . . 95
4.1.4.189 setAccelerometerRange . . . 95
4.1.4.190 setAvgPercent . . . 95
4.1.4.191 setAxisDirections . . . 96
4.1.4.192 setButtonGyroDisableLength . . . 96
4.1.4.193 setChannel . . . 96
4.1.4.194 setClock . . . 96
4.1.4.195 setClockc6 . . . 97
4.1.4.196 setClockSpeed . . . 97
4.1.4.197 setCompassCalibrationParam . . . 97
4.1.4.198 setCompassCalibrationParamf9f3 . . . 98
4.1.4.199 setCompassEnabled . . . 98
4.1.4.200 setCompassRange . . . 98
4.1.4.201 setConfidenceRhoModeAccel . . . 99
4.1.4.202 setConfidenceRhoModeCompass . . . 99
4.1.4.203 setFilterMode . . . 100
4.1.4.204 setGlobalAxis . . . 100
4.1.4.205 setGyroCalibrationParam . . . 101
4.1.4.206 setGyroCalibrationParamf9f3 . . . 101
4.1.4.207 setGyroEnabled . . . 101
4.1.4.208 setGyroRange . . . 102
4.1.4.209 setHIDUpdateRate . . . 102
4.1.4.210 setInterruptType . . . 102
4.1.4.211 setJoystickEnabled . . . 103
4.1.4.212 setJoystickLogicalID . . . 103
4.1.4.213 setJoystickMousePresent . . . 103
4.1.4.214 setLEDColor . . . 104
4.1.4.215 setLEDColorf3 . . . 104
4.1.4.216 setLEDMode . . . 104
4.1.4.217 setLookupTblVertVal . . . 105

4.1.4.218 setLookupTblVertValf3 . . . 105
4.1.4.219 setMouseEnabled . . . 105
4.1.4.220 setMouseLogicalID . . . 106
4.1.4.221 setMouseRelative . . . 106
4.1.4.222 setMultiRefChkVecAccelerometer . . . 106
4.1.4.223 setMultiRefChkVecAccelerometerf3 . . . 107
4.1.4.224 setMultiRefChkVecCompass . . . 107
4.1.4.225 setMultiRefChkVecCompassf3 . . . 107
4.1.4.226 setMultiRefResolution . . . 108
4.1.4.227 setMultiRefVecAccelerometer . . . 108
4.1.4.228 setMultiRefVecAccelerometerf3 . . . 108
4.1.4.229 setMultiRefVecCompass . . . 109
4.1.4.230 setMultiRefVecCompassf3 . . . 109
4.1.4.231 setMultiRefVecZero . . . 109
4.1.4.232 setMultiRefWeightPwr . . . 109
4.1.4.233 setOrientationButton . . . 110
4.1.4.234 setOversampleRate . . . 110
4.1.4.235 setPanID . . . 110
4.1.4.236 setPhysicalButton . . . 111
4.1.4.237 setRefVecAccelerometer . . . 111
4.1.4.238 setRefVecAccelerometerf3 . . . 111
4.1.4.239 setRefVecCompass . . . 112
4.1.4.240 setRefVecCompassf3 . . . 112
4.1.4.241 setRefVecCurrentOrient . . . 112
4.1.4.242 setRefVecMode . . . 112
4.1.4.243 setRunningAvgMode . . . 113
4.1.4.244 setScreenPointAxis . . . 113
4.1.4.245 setSerialNumber . . . 114
4.1.4.246 setShakeButton . . . 114
4.1.4.247 setStaticRhoModeAccel . . . 115
4.1.4.248 setStaticRhoModeCompass . . . 115
4.1.4.249 setTareCurrentOrient . . . 115
4.1.4.250 setTareMatrix . . . 115
4.1.4.251 setTareMatrixf9 . . . 116
4.1.4.252 setTareQuaternion . . . 116
4.1.4.253 setTareQuaternionf4 . . . 116
4.1.4.254 setToFirmwareMode . . . 117
4.1.4.255 setUARTRate . . . 117
4.1.4.256 setUpdateRate . . . 117
4.1.4.257 setUSBMode . . . 117

4.1.4.258 setWirelessAddress 118
4.1.4.259 setWirelessHWID 118
4.1.4.260 setWirelessRetries 118
4.1.4.261 setWriteStart 119
4.1.4.262 softwareReset 119
4.1.4.263 startLogSession 119
4.1.4.264 stopAsynchronous 119
4.1.4.265 stopLogSession 120
4.1.4.266 updateAsynchData 120
4.1.4.267 writeEncryptedData 120
4.1.4.268 writeUnencryptedData 120

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

AxisDirections
: Stores a full configuration of a 3-Space sensor's 'Axis Directions'. This includes both remapping axes of rotation and negating the mapped axes 5

BootloaderData
: A storage structure for information about the state of a 3-Space device in 'bootloader' mode . . 5

ButtonState
: Stores the state of the physical buttons on a 3-Space sensor 6

ComInfo
: This structure stores an expanded collection attributes of a serial port (virtual or otherwise) and the possible 3-Space device connected to it 6

ComPort
: This structure stores all attributes of a serial port (virtual or otherwise) relevant to the API . . . 7

SensorData
: A storage structure for a dump of all data from a 3-Space Sensor's on-board sensors 7

TSS_AxisAngle
: A storage structure for orientations expressed as an Axis/Angle pair 8

TSS_Clock
: Stores a timestamp retrieved from 3-Space Sensor Data-Logging units 9

TSS_Color
: Stores a 3-channel color 10

TSS_Euler
: A storage structure for orientations expressed as rotations about the three cartesian axes . . . 10

TSS_Matrix
: A storage structure for orientations expressed as a 3x3 TSS_Matrix 11

TSS_Quaternion
: A storage structure for orientations expressed in fourth dimensional space 12

TSS_Vector
: A storage structure for three dimensional coordinates or values 12

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

include/yei_threespace_api.h 13

Chapter 3

Data Structure Documentation

3.1 AxisDirections Struct Reference

Stores a full configuration of a 3-Space sensor's 'Axis Directions'. This includes both remapping axes of rotation and negating the mapped axes.

`#include <yei_threespace_api.h>`

Data Fields

- unsigned char axis_order

  The mapping of axes from a 3-Space sensor's Natural Axes to the sensor's Output Axes. This is easily assigned by using the Axis_Order enum.
- boolean neg_x

  Whether to negate the X-Axis of the sensor's Output Axis.
- boolean neg_y

  Whether to negate the Y-Axis of the sensor's Output Axis.
- boolean neg_z

  Whether to negate the Z-Axis of the sensor's Output Axis.

3.1.1 Detailed Description

Stores a full configuration of a 3-Space sensor's 'Axis Directions'. This includes both remapping axes of rotation and negating the mapped axes.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.2 BootloaderData Struct Reference

A storage structure for information about the state of a 3-Space device in 'bootloader' mode.

`#include <yei_threespace_api.h>`

Data Fields

- unsigned int jtag_id

TODO: Write description...

- unsigned int program_offset

  TODO: Write description...
- unsigned int flash_size

  TODO: Write description...
- unsigned short page_size

  TODO: Write description...
- unsigned int user_page_offset

  TODO: Write description...
- unsigned short user_page_size

  TODO: Write description...
- unsigned short app_firmware_ver

  TODO: Write description...
- unsigned short firmware_ver

  TODO: Write description...

3.2.1 Detailed Description

A storage structure for information about the state of a 3-Space device in 'bootloader' mode.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.3 ButtonState Struct Reference

Stores the state of the physical buttons on a 3-Space sensor.

`#include <yei_threespace_api.h>`

Data Fields

- boolean left_pressed

  True if the left button is pressed. False if not.
- boolean right_pressed

  True if the right button is pressed. False if not.

3.3.1 Detailed Description

Stores the state of the physical buttons on a 3-Space sensor.

Designation of "left" and "right" are based on the assumed that the sensor' side with physical buttons is facing towards the point of reference with the LED on that side pointed upward.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.4 ComInfo Struct Reference

This structure stores an expanded collection attributes of a serial port (virtual or otherwise) and the possible 3-Space device connected to it.

`#include <yei_threespace_api.h>`

Data Fields

- char friendly_name [256]

  The human-friendly name for the serial port.
- TSS_Type sensor_type

  The type of 3-Space device connected through the serial port.
- unsigned int serial_number

  The serial number for the 3-Space device connected through the serial port.
- char software_ver [256]

  The version of the firmware installed on the connected 3-Space device.
- char hardware_ver [256]

  The hardware revision and type of the connected 3-Space device.
- boolean is_bootloader

  Expressing whether the connected device is in its 'bootloader' mode or not.

3.4.1 Detailed Description

This structure stores an expanded collection attributes of a serial port (virtual or otherwise) and the possible 3-Space device connected to it.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.5 ComPort Struct Reference

This structure stores all attributes of a serial port (virtual or otherwise) relevant to the API.

`#include <yei_threespace_api.h>`

Data Fields

- char com_port [64]

  The system name for the serial port.
- char friendly_name [256]

  The human-friendly name for the serial port.
- TSS_Type sensor_type

  The type of 3-Space device connected through the serial port.

3.5.1 Detailed Description

This structure stores all attributes of a serial port (virtual or otherwise) relevant to the API.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.6 SensorData Struct Reference

A storage structure for a dump of all data from a 3-Space Sensor's on-board sensors.

`#include <yei_threespace_api.h>`

Data Fields

- float gyro_x

  The rate of rotation about the gyro's X-Axis.
- float gyro_y

  The rate of rotation about the gyro's Y-Axis.
- float gyro_z

  The rate of rotation about the gyro's Z-Axis.
- float accelerometer_x

  The amount of acceleration experienced on the accelerometer's X-Axis.
- float accelerometer_y

  The amount of acceleration experienced on the accelerometer's Y-Axis.
- float accelerometer_z

  The amount of acceleration experienced on the accelerometer's Z-Axis.
- float compass_x

  The x component of the magnetic north vector.
- float compass_y

  The y component of the magnetic north vector.
- float compass_z

  The z component of the magnetic north vector.

3.6.1 Detailed Description

A storage structure for a dump of all data from a 3-Space Sensor's on-board sensors.

The three on-board sensors used on a 3-Space Sensor are a gyroscope, and accelerometer, and a compass. The gyroscope expresses rates of rotations about each of its major axes. The accelerometer expresses any experienced accleration about each of its major axes. The compass expresses a three dimensional vector pointed towards magnetic north.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.7 TSS_AxisAngle Struct Reference

A storage structure for orientations expressed as an Axis/Angle pair.

```
#include <yei_threespace_api.h>
```

Data Fields

- float x

  The x element of the stored axis.
- float y

  The y element of the stored axis.
- float z

  The z element of the stored axis.
- float angle

  The angle rotated about the stored axis.

3.7.1 Detailed Description

A storage structure for orientations expressed as an Axis/Angle pair.

The stored orientation holds a normalized three dimensional vector and an angle in radians.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.8 TSS_Clock Struct Reference

Stores a timestamp retrieved from 3-Space Sensor Data-Logging units.

`#include <yei_threespace_api.h>`

Data Fields

- unsigned char month
  The Month recorded by the timestamp.
- unsigned char day
  The Day recorded by the timestamp.
- unsigned char year
  The Year recorded by the timestamp.
- unsigned char hour
  The Hour recorded by the timestamp.
- unsigned char minute
  The Minute recorded by the timestamp.
- unsigned char second
  The Second recorded by the timestamp.

3.8.1 Detailed Description

Stores a timestamp retrieved from 3-Space Sensor Data-Logging units.

3.8.2 Field Documentation

3.8.2.1 unsigned char TSS_Clock::day

The Day recorded by the timestamp.

Expected values can range from 0 - 39.

3.8.2.2 unsigned char TSS_Clock::hour

The Hour recorded by the timestamp.

Expected values can range from 0 - 23.

3.8.2.3 unsigned char TSS_Clock::minute

The Minute recorded by the timestamp.

Expected values can range from 0 - 59.

3.8.2.4 unsigned char TSS_Clock::month

The Month recorded by the timestamp.

Expected values can range from 1 - 12. 1 = January, 2 = February, ...

3.8.2.5 unsigned char TSS_Clock::second

The Second recorded by the timestamp.

Expected values can range from 0 - 59.

3.8.2.6 unsigned char TSS_Clock::year

The Year recorded by the timestamp.

Expected values can range from 0 - 159.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.9 TSS_Color Struct Reference

Stores a 3-channel color.

`#include <yei_threespace_api.h>`

Data Fields

- float r

  The red channel of the color. May be between 0.0 and 1.0.
- float g

  The green channel of the color. May be between 0.0 and 1.0.
- float b

  The blue channel of the color. May be between 0.0 and 1.0.

3.9.1 Detailed Description

Stores a 3-channel color.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.10 TSS_Euler Struct Reference

A storage structure for orientations expressed as rotations about the three cartesian axes.

`#include <yei_threespace_api.h>`

Data Fields

- float x
  The x component of the TSS_Euler.
- float y
  The y component of the TSS_Euler.
- float z
  The z component of the TSS_Euler.

3.10.1 Detailed Description

A storage structure for orientations expressed as rotations about the three cartesian axes.

TSS_Euler values returned from API calls always return with the assumed rotation order X (Pitch), Y (Yaw), Z (Roll).

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.11 TSS_Matrix Struct Reference

A storage structure for orientations expressed as a 3x3 TSS_Matrix.

```
#include <yei_threespace_api.h>
```

Data Fields

- float m00
  The value in row 0 column 0.
- float m01
  The value in row 0 column 1.
- float m02
  The value in row 0 column 2.
- float m10
  The value in row 1 column 0.
- float m11
  The value in row 1 column 1.
- float m12
  The value in row 1 column 2.
- float m20
  The value in row 2 column 0.
- float m21
  The value in row 2 column 1.
- float m22
  The value in row 2 column 2.

3.11.1 Detailed Description

A storage structure for orientations expressed as a 3x3 TSS_Matrix.

Matricies returned from API calls are always in row-major. Fields for the structure are named based on the following template: m[ROW][COLUMN]. This means that matrix field m12 corresponds to the value in row 1 column 2.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.12 TSS_Quaternion Struct Reference

A storage structure for orientations expressed in fourth dimensional space.

`#include <yei_threespace_api.h>`

Data Fields

- float x
  The x component of the quaternion.
- float y
  The y component of the quaternion.
- float z
  The z component of the quaternion.
- float w
  The z component of the quaternion.

3.12.1 Detailed Description

A storage structure for orientations expressed in fourth dimensional space.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

3.13 TSS_Vector Struct Reference

A storage structure for three dimensional coordinates or values.

`#include <yei_threespace_api.h>`

Data Fields

- float x
  The first component of the vector.
- float y
  The second component of the vector.
- float z
  The third component of the vector.

3.13.1 Detailed Description

A storage structure for three dimensional coordinates or values.

The documentation for this struct was generated from the following file:

- include/yei_threespace_api.h

Chapter 4

File Documentation

4.1 include/yei_threespace_api.h File Reference

Data Structures

- struct ComPort

  This structure stores all attributes of a serial port (virtual or otherwise) relevant to the API.

- struct ComInfo

  This structure stores an expanded collection attributes of a serial port (virtual or otherwise) and the possible 3-Space device connected to it.

- struct AxisDirections

  Stores a full configuration of a 3-Space sensor's 'Axis Directions'. This includes both remapping axes of rotation and negating the mapped axes.

- struct ButtonState

  Stores the state of the physical buttons on a 3-Space sensor.

- struct TSS_Color

  Stores a 3-channel color.

- struct TSS_Clock

  Stores a timestamp retrieved from 3-Space Sensor Data-Logging units.

- struct TSS_Vector

  A storage structure for three dimensional coordinates or values.

- struct TSS_Quaternion

  A storage structure for orientations expressed in fourth dimensional space.

- struct TSS_Euler

  A storage structure for orientations expressed as rotations about the three cartesian axes.

- struct TSS_Matrix

  A storage structure for orientations expressed as a 3x3 TSS_Matrix.

- struct TSS_AxisAngle

  A storage structure for orientations expressed as an Axis/Angle pair.

- struct SensorData

  A storage structure for a dump of all data from a 3-Space Sensor's on-board sensors.

- struct BootloaderData

  A storage structure for information about the state of a 3-Space device in 'bootloader' mode.

Defines

- #define **EXPORT** __declspec(dllexport)
- #define **NULL** 0
- #define **True** 1
- #define **False** 0
- #define **INF_DURATION** 0xffff
- #define **NO_DEVICE_ID** 0x800000
- #define **BOOTLOADER_ID** 0x1000000
- #define **DONGLE_ID** 0x2000000
- #define **SENSOR_ID** 0x4000000
- #define **EMBEDDED_ID** 0x8000000
- #define **WIRELESS_ID** 0x10000000
- #define **WIRELESS_W_ID** 0x20000000
- #define **DATALOGGER_ID** 0x40000000
- #define **BLUETOOTH_ID** 0x80000000
- #define **NO_DONGLE_ID** 0xfd000000
- #define **ALL_SENSORS_ID** 0xff000000
- #define **TSS_FIND_ALL** -1
- #define **TSS_FIND_BTL** 1
- #define **TSS_FIND_USB** 2
- #define **TSS_FIND_DNG** 4
- #define **TSS_FIND_WL** 8
- #define **TSS_FIND_EM** 16
- #define **TSS_FIND_DL** 32
- #define **TSS_FIND_BT** 64

Typedefs

- typedef unsigned char **boolean**
- typedef unsigned int **TSS_ID**
- typedef enum TSS_Type TSS_Type

  An enum expressing the different types of 3-Space Sensors.
- typedef enum TSS_Error TSS_Error

  An enum expressing the different types of errors a 3-Space API call can return.
- typedef enum Axis_Order Axis_Order

  An enum expressing the different possible configurations of axis assignment between the axes of rotation the physical 3-Space sensor experiences (Natural Axes) and the axes of rotation depiced by the sensor's output data (Output Axes).
- typedef struct ComPort ComPort

  This structure stores all attributes of a serial port (virtual or otherwise) relevant to the API.
- typedef struct ComInfo ComInfo

  This structure stores an expanded collection attributes of a serial port (virtual or otherwise) and the possible 3-Space device connected to it.
- typedef struct AxisDirections AxisDirections

  Stores a full configuration of a 3-Space sensor's 'Axis Directions'. This includes both remapping axes of rotation and negating the mapped axes.
- typedef struct ButtonState ButtonState

  Stores the state of the physical buttons on a 3-Space sensor.
- typedef struct TSS_Color TSS_Color

  Stores a 3-channel color.
- typedef struct TSS_Clock TSS_Clock

  Stores a timestamp retrieved from 3-Space Sensor Data-Logging units.

- typedef struct TSS_Vector TSS_Vector

  A storage structure for three dimensional coordinates or values.
- typedef struct TSS_Quaternion TSS_Quaternion

  A storage structure for orientations expressed in fourth dimensional space.
- typedef struct TSS_Euler TSS_Euler

  A storage structure for orientations expressed as rotations about the three cartesian axes.
- typedef struct TSS_Matrix TSS_Matrix

  A storage structure for orientations expressed as a 3x3 TSS_Matrix.
- typedef struct TSS_AxisAngle TSS_AxisAngle

  A storage structure for orientations expressed as an Axis/Angle pair.
- typedef struct SensorData SensorData

  A storage structure for a dump of all data from a 3-Space Sensor's on-board sensors.
- typedef struct BootloaderData BootloaderData

  A storage structure for information about the state of a 3-Space device in 'bootloader' mode.

Enumerations

- enum TSS_Type {
  TSS_BTL, **TSS_USB**, **TSS_DNG**, **TSS_WL**,
  TSS_WL_W, **TSS_EM**, **TSS_DL**, **TSS_BT**,
  TSS_UNKNOWN }

  An enum expressing the different types of 3-Space Sensors.
- enum TSS_Error {
  TSS_NO_ERROR = 0, TSS_INVALID_COMMAND, TSS_INVALID_ID, TSS_INVALID_IDX,
  TSS_WARNING_OLD_FIRMWARE, TSS_ERROR_OLD_FIRMWARE, TSS_ERROR_USB_CONNECTIO-
  N, TSS_ERROR_WIR_CONNECTION,
  TSS_ERROR_WRITE, TSS_ERROR_READ, TSS_ERROR_NO_SENSOR_FOUND }

  An enum expressing the different types of errors a 3-Space API call can return.
- enum Axis_Order {
  XYZ = 0, XZY, YXZ, YZX,
  ZXY, ZYX }

  An enum expressing the different possible configurations of axis assignment between the axes of rotation the physical 3-Space sensor experiences (Natural Axes) and the axes of rotation depiced by the sensor's output data (Output Axes).

Functions

- EXPORT void getAvailableComPorts (ComPort **tss_port_list, ComPort **ukn_port_list, unsigned int *tss-
  _list_size, unsigned int *ukn_list_size, int filter)

  Scans the host computer for all serial ports (virtual or physical). Ports returned are sorted into two lists based on whether the driver creating the port was a 3-Space driver.
- EXPORT TSS_Error getFirstAvailableTSSComPort (ComPort *tss_port, int filter)

  Scans the host computer for the first serial port it finds with a sensor (virtual or physical). Port is returned by tss_port. Calling getAvailableComPorts or getFirstAvailableTSSComPort will reset the iteration through the sensors.
- EXPORT TSS_Error getNextAvailableTSSComPort (ComPort *tss_port)

  Call after getFirstAvailableTSSComPort to get more sensors. Port is returned by tss_port. getFirstAvailableTSSCom-Port must be called before calling getNextAvailableTSSComPort Calling getAvailableComPorts or getFirstAvailableT-SSComPort will reset the iteration through the sensors.
- EXPORT TSS_Error getFirstAvailableUnknownComPort (ComPort *ukn_port)

  Scans the host computer for the first serial port it finds with an unknown device (virtual or physical). Port is returned by ukn_port. Calling getAvailableComPorts or getFirstAvailableUnknownComPort will reset the iteration through the unknown ports.
- EXPORT TSS_Error getNextAvailableUnknownComPort (ComPort *ukn_port)

Call after getFirstAvailableUnknownComPort to get more sensors. Port is returned by tss_port. getFirstAvailableUnknownComPort must be called before calling getNextAvailableUnknownComPort Calling getAvailableComPorts or getFirstAvailableUnknownComPort will reset the iteration through the unknown ports.

- EXPORT void getComPortInfo (const char *com_port, ComInfo *com_info, boolean poll_device)

  Interrogates an indicated serial port for information about the device connected to it. Optionaly, the port may be polled to gain further information about the connected device. Polling a port may cause undesired effects if the device connected to the port is not a 3-Space device, as bytes will be written to the serial port. Its suggested that any polling done in an application be an optional action invoked by the user with proper warnings in place.
- EXPORT TSS_ID createTSDevice (const ComPort com_port)

  Given a serial port, the type of 3-Space device is determined from the passed in structure and the device is connected to. An identifier for the device is returned.
- EXPORT TSS_ID createTSDeviceStr (const char *com_port, TSS_Type type)

  Given a serial port and the type of 3-Space device, the device is connected to. An identifier for the device is returned.
- EXPORT TSS_Error closeTSDevice (TSS_ID id)

  Disconnects the 3-Space device associated with the inputted ID. If connected through a serial port, that port is closed.
- EXPORT boolean isConnected (TSS_ID id, boolean try_reconnect)

  Checks if a 3-Space device is currently connected to the computer.
- EXPORT TSS_Error getSerialNumberHex (TSS_ID id, char *hex_serial)

  Retrieves a hexidecimal string representation of the 3-Space device's serial number (matching the representation on the case of the device).
- EXPORT TSS_Error getDeviceType (TSS_ID id, TSS_Type *device_type)

  Retrieves the type of 3-Space device connected as a string.
- EXPORT TSS_Error getFriendlyName (TSS_ID id, char *friendly_name)

  Retrieves the friendly name of the device's serial port as reported by the port's driver.
- EXPORT TSS_Error addSensorToDongle (TSS_ID id, unsigned char idx, unsigned int hw_id, AxisDirections *axis_dir, TSS_ID *wl_id)

  Registers a 3-Space Wireless Sensor with a 3-Space Dongle and connects the wireless sensor.
- EXPORT TSS_Error getSensorFromDongle (TSS_ID id, unsigned int idx, TSS_ID *wl_id)

  Retrieves the unique identifier for a 3-Space Wireless Sensor already registered with a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredOrientQuat (TSS_ID id, TSS_Quaternion *quat)

  Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a TSS_Quaternion. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredOrientQuatf4 (TSS_ID id, float *quat)

  Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a TSS_Quaternion. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredOrientEuler (TSS_ID id, TSS_Euler *euler)

  Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a set of TSS_Euler angles. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredOrientEulerf3 (TSS_ID id, float *euler)

  Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a set of TSS_Euler angles. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredOrientMat (TSS_ID id, TSS_Matrix *mat)

  Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredOrientMatf9 (TSS_ID id, float *mat)

  Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredOrientAxisAngle (TSS_ID id, TSS_AxisAngle *axis_ang)

  Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as an Axis Angle. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredOrientAxisAnglef4 (TSS_ID id, float *axis_ang)

  Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as an Axis Angle. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredOrientFwdDwn (TSS_ID id, TSS_Vector *forward, TSS_Vector *down)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a pair of two vectors (A Forward vector and a Down vector). This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error getFiltTaredOrientFwdDwnf3f3 (TSS_ID id, float *forward, float *down)

  Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a pair of two vectors (A Forward vector and a Down vector). This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltGyroRate (TSS_ID id, TSS_Quaternion *quat)

  Returns the gyro rates as determined by taking the difference between the current orientation and the previous one. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltGyroRatef4 (TSS_ID id, float *quat)

  Returns the gyro rates as determined by taking the difference between the current orientation and the previous one. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltOrientQuat (TSS_ID id, TSS_Quaternion *quat)

  Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a TSS_Quaternion. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltOrientQuatf4 (TSS_ID id, float *quat)

  Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a TSS_Quaternion. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltOrientEuler (TSS_ID id, TSS_Euler *euler)

  Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a set of TSS_Euler angles. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltOrientEulerf3 (TSS_ID id, float *euler)

  Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a set of TSS_Euler angles. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltOrientMat (TSS_ID id, TSS_Matrix *mat)

  Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltOrientMatf9 (TSS_ID id, float *mat)

  Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltOrientAxisAngle (TSS_ID id, TSS_AxisAngle *axis_ang)

  Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as an Axis Angle. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltOrientAxisAnglef4 (TSS_ID id, float *axis_ang)

  Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as an Axis Angle. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltOrientFwdDwn (TSS_ID id, TSS_Vector *forward, TSS_Vector *down)

  Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a pair of two vectors (A Forward vector and a Down vector). This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltOrientFwdDwnf3f3 (TSS_ID id, float *forward, float *down)

  Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a pair of two vectors (A Forward vector and a Down vector). This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredFwdDwnVecSensFrame (TSS_ID id, TSS_Vector *forward, TSS_Vector *down)

  Returns the forward and down vectors as determined by the Kalman filter, relative to the tare orientation. This version returns the data as the forward and down vectors of the coordinate system defined by the sensor reference frame. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltTaredFwdDwnVecSensFramef3f3 (TSS_ID id, float *forward, float *down)

  Returns the forward and down vectors as determined by the Kalman filter, relative to the tare orientation. This version returns the data as the forward and down vectors of the coordinate system defined by the sensor reference frame. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltNorthEarthVecSensFrame (TSS_ID id, TSS_Vector *north, TSS_Vector *earth)

  Returns the North and Earth vectors as determeined by the Kalman filter, relative to the global sensor references. Returned vectors are in the coordinate system defined by the sensor reference frame. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFiltNorthEarthVecSensFramef3f3 (TSS_ID id, float *north, float *earth)

Returns the North and Earth vectors as determeined by the Kalman filter, relative to the global sensor references. Returned vectors are in the coordinate system defined by the sensor reference frame. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setInterruptType (TSS_ID id, unsigned char mode, unsigned char pin)

  Configures how the sensor will generate interrupts.. This call will error if the inputted id is not for a 3-Space Embedded Sensor.
- EXPORT TSS_Error getInterruptType (TSS_ID id, unsigned char *mode, unsigned char *pin)

  Returns how the sensor currently generates interrupts. This call will error if the inputted id is not for a 3-Space Embedded Sensor.
- EXPORT TSS_Error getInterruptStatus (TSS_ID id, unsigned char *status)

  Read the current interrupt state. Calling this command while interrupts are in mode '2' (see definition for setInterruptType for more details) will cause the interrupt pin to return to high. This call will error if the inputted id is not for a 3-Space Embedded Sensor.
- EXPORT TSS_Error getAllSensorsNormalized (TSS_ID id, SensorData *data)

  Returns the normalized data of the 3-Space Sensor's on-board sensors. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAllSensorsNormalizedf9 (TSS_ID id, float *data)

  Returns the normalized data of the 3-Space Sensor's on-board sensors. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getGyroNormalized (TSS_ID id, TSS_Vector *gyro)

  Returns the normalized gyro rates of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getGyroNormalizedf3 (TSS_ID id, float *gyro)

  Returns the normalized gyro rates of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAccelerometerNormalized (TSS_ID id, TSS_Vector *accelerometer)

  Returns the normalized accelerometer vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAccelerometerNormalizedf3 (TSS_ID id, float *accelerometer)

  Returns the normalized accelerometer vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getCompassNormalized (TSS_ID id, TSS_Vector *compass)

  Returns the normalized compass vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getCompassNormalizedf3 (TSS_ID id, float *compass)

  Returns the normalized compass vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getTemperatureCelsius (TSS_ID id, float *temp_c)

  Returns the temperature of the sensor in Celsius. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getTemperatureFahrenheit (TSS_ID id, float *temp_f)

  Returns the temperature of the sensor in Fahrenheit. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getConfidence (TSS_ID id, float *confidence)

  Returns a value indicating how much the compass and accelerometer are to be trusted to be unit vectors pointing in the proper directions at the moment. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAccelerometerUnfiltered (TSS_ID id, TSS_Vector *accelerometer)

  Returns the unfiltered (but compensated) data of the 3-Space Sensor's on-board accelerometer. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAccelerometerUnfilteredf3 (TSS_ID id, float *accelerometer)

  Returns the unfiltered (but compensated) data of the 3-Space Sensor's on-board accelerometer. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getCompassUnfiltered (TSS_ID id, TSS_Vector *compass)

  Returns the unfiltered (but compensated) data of the 3-Space Sensor's on-board compass. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getCompassUnfilteredf3 (TSS_ID id, float *compass)

  Returns the unfiltered (but compensated) data of the 3-Space Sensor's on-board compass. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error enableMassStorageMode (TSS_ID id)

  When enabled, the 3-Space Sensor acts as a mass storage device on the host system. This is useful for reading/writing log files to the sensor. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.
- EXPORT TSS_Error disableMassStorageMode (TSS_ID id)

  When disabled, the 3-Space Sensor's mass storage features are hidden from the host machine. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.
- EXPORT TSS_Error formatInitSDCard (TSS_ID id)

  The SD card contained within the 3-Space Sensor is formatted to the FAT-32 file system and initialized for writing/reading. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.
- EXPORT TSS_Error startLogSession (TSS_ID id)

  Begins a data logging session based on the contents of the capture.cfg file.
- EXPORT TSS_Error stopLogSession (TSS_ID id)

  Stops any active capture sessions. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.
- EXPORT TSS_Error setClock (TSS_ID id, TSS_Clock clock_data)

  Sets the current data and time of the 3-Space Sensor's internal clock to the inputted date and time. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.
- EXPORT TSS_Error setClockc6 (TSS_ID id, unsigned char *clock_data)

  Sets the current data and time of the 3-Space Sensor's internal clock to the inputted date and time. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.
- EXPORT TSS_Error getClock (TSS_ID id, TSS_Clock *clock_data)

  Gets the current data and time of the 3-Space Sensor's internal clock. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.
- EXPORT TSS_Error getClockc6 (TSS_ID id, unsigned char *clock_data)

  Gets the current data and time of the 3-Space Sensor's internal clock. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor. This array will be formatted as: month, day, year, hour, minute, second.
- EXPORT TSS_Error getAllSensorsRaw (TSS_ID id, SensorData *data)

  Returns the unaltered data of the 3-Space Sensor's on-board sensors. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAllSensorsRawf9 (TSS_ID id, float *data)

  Returns the unaltered data of the 3-Space Sensor's on-board sensors. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getGyroRaw (TSS_ID id, TSS_Vector *gyro)

  Returns the unaltered gyro rates of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getGyroRawf3 (TSS_ID id, float *gyro)

  Returns the unaltered gyro rates of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error writeUnencryptedData (TSS_ID id, unsigned char *data, unsigned int data_size)

  Writes unencrypted data to the 3-Space Sensor. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.
- EXPORT TSS_Error getAccelerometerRaw (TSS_ID id, TSS_Vector *accelerometer)

  Returns the unaltered data of the 3-Space Sensor's on-board accelerometer. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAccelerometerRawf3 (TSS_ID id, float *accelerometer)

  Returns the unaltered data of the 3-Space Sensor's on-board accelerometer. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setToFirmwareMode (TSS_ID id)

  Resets the device to use firmware loaded to the 3-Space device. Will remain in bootloader mode if the firmware was corrupted. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.
- EXPORT TSS_Error getCompassRaw (TSS_ID id, TSS_Vector *compass)

  Returns the unaltered compass vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getCompassRawf3 (TSS_ID id, float *compass)

  Returns the unaltered compass vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error writeEncryptedData (TSS_ID id, unsigned char *data, unsigned int data_size)

Writes encrypted data to the 3-Space Sensor. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

- EXPORT TSS_Error finishWrite (TSS_ID id)

  Tells the 3-Space device to stop writting. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

- EXPORT TSS_Error getInformation (TSS_ID id, BootloaderData *data)

  Gets information about the 3-Space device in bootloader mode. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

- EXPORT TSS_Error setSerialNumber (TSS_ID id, unsigned int serial)

  Sets the serial number for the 3-Space device in bootloader mode. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

- EXPORT TSS_Error isFirmwareValid (TSS_ID id, boolean *is_valid)

  Checks the firmware loaded into the 3-Space device is valid or not. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

- EXPORT TSS_Error setWriteStart (TSS_ID id, unsigned int start_addr)

  Sets the memory address for writting firmware to the 3-Space device in bootloader mode. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

- EXPORT TSS_Error setTareCurrentOrient (TSS_ID id)

  Sets current filtered orientation derived from its detected gravity and north vectors as the tare orientation. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setTareQuaternion (TSS_ID id, TSS_Quaternion quat)

  Sets current filtered orientation to a passed in quaternion. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setTareQuaternionf4 (TSS_ID id, float *quat)

  Sets current filtered orientation to a passed in quaternion. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setTareMatrix (TSS_ID id, TSS_Matrix mat)

  Sets current filtered orientation to a passed in rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setTareMatrixf9 (TSS_ID id, float *mat)

  Sets current filtered orientation to a passed in rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setStaticRhoModeAccel (TSS_ID id, float val)

  Sets the rho mode for the accelerometer to static, using the given value as rho. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setConfidenceRhoModeAccel (TSS_ID id, float min, float max)

  Sets the rho mode for the accelerometer to confidence, using the given values as the min and the max. The confidence factor will be used in real time to interpolate between the min and max to determine what rho value is used. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setStaticRhoModeCompass (TSS_ID id, float val)

  Sets the rho mode for the compass to static, using the given value as rho. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setConfidenceRhoModeCompass (TSS_ID id, float min, float max)

  Sets the rho mode for the compass to confidence, using the given values as the min and the max. The confidence factor will be used in real time to interpolate between the min and max to determine what rho value is used. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setUpdateRate (TSS_ID id, unsigned int rate)

  Sets the target update rate to the given rate. If the filter takes less time to update during a given frame than this rate specifies, the frame will be padded out to take the specified amount of time. If the filter takes more time to update than this rate, this rate will be ignored. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setRefVecCurrentOrient (TSS_ID id)

  Uses the current tared orientation to set up the reference vector for the nearest orthogonal orientation. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setRefVecMode (TSS_ID id, unsigned char mode)

  Sets reference vector mode to either 'single static', 'single auto', 'single auto continuous', or 'multi'.

- EXPORT TSS_Error setOversampleRate (TSS_ID id, unsigned char rate)

Sets oversample rate of data reads from the 3-Space Sensor's on-board sensors. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setGyroEnabled (TSS_ID id, boolean do_enable)

  Enables or disables the gyros (will be replaced with a still gyro reading if disabled). This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setAccelerometerEnabled (TSS_ID id, boolean do_enable)

  Enables or disables the accelerometer (the filter will not use this data if disabled). This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setCompassEnabled (TSS_ID id, boolean do_enable)

  Enables or disables the compass (the filter will not use this data if disabled). This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMultiRefVecZero (TSS_ID id)

  Resets all reference vectors in the multi reference scheme to zero. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMultiRefResolution (TSS_ID id, unsigned char num_division, unsigned char num_vecs)

  Sets number of cell divisions and number of nearby vectors per cell for the multi reference lookup table.
- EXPORT TSS_Error setMultiRefVecCompass (TSS_ID id, unsigned char idx, TSS_Vector vec)

  Directly set compass multi reference vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMultiRefVecCompassf3 (TSS_ID id, unsigned char idx, float *vec)

  Directly set compass multi reference vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMultiRefChkVecCompass (TSS_ID id, unsigned char idx, TSS_Vector vec)

  Directly set compass multi reference check vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMultiRefChkVecCompassf3 (TSS_ID id, unsigned char idx, float *vec)

  Directly set compass multi reference check vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMultiRefVecAccelerometer (TSS_ID id, unsigned char idx, TSS_Vector vec)

  Directly set accelerometer multi reference vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMultiRefVecAccelerometerf3 (TSS_ID id, unsigned char idx, float *vec)

  Directly set accelerometer multi reference vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMultiRefChkVecAccelerometer (TSS_ID id, unsigned char idx, TSS_Vector vec)

  Directly set accelerometer multi reference check vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMultiRefChkVecAccelerometerf3 (TSS_ID id, unsigned char idx, float *vec)

  Directly set accelerometer multi reference check vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setAxisDirections (TSS_ID id, AxisDirections axis_dir)

  Sets the axis direction configuration for a 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setAvgPercent (TSS_ID id, float val)

  Sets what percentage of running average to use on the sensor's orientation.
- EXPORT TSS_Error setRefVecCompass (TSS_ID id, TSS_Vector vec)

  Sets the static compass reference vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setRefVecCompassf3 (TSS_ID id, float *vec)

  Sets the static compass reference vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setRefVecAccelerometer (TSS_ID id, TSS_Vector vec)

  Sets the static accelerometer reference vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setRefVecAccelerometerf3 (TSS_ID id, float *vec)

  Sets the static accelerometer reference vector. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error resetKalmanFilter (TSS_ID id)

  Resets Kalman filter's covariance and state matrices. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setAccelerometerRange (TSS_ID id, unsigned char range)

  Set which range of accelerometer data to use. Higher ranges can detect and report larger accelerations, but are not as accurate for smaller ones. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMultiRefWeightPwr (TSS_ID id, float pow)

  Set weighting power for multi reference vector weights. Multi reference vector weights are all raised to the weight power before they are summed and used in the calculation for the final reference vector. Setting this value nearer to 0 will cause the reference vectors to overlap more, and setting it nearer to infinity will cause the reference vectors to influence a smaller set of orientations. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setFilterMode (TSS_ID id, unsigned char mode)

  Used to disable the orientation filter or set the orientation filter mode. Changing this parameter can be useful for tuning filter-performance versus orientation-update rates. When using the sensor as an IMU, it will improve performance to disable the orientation filter. When using as an AHRS, where orientation is needed, full-Kalman, gyroless-filtered, or fast-filtered should be used. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setRunningAvgMode (TSS_ID id, unsigned char mode)

  Selects the mode that the running-average method uses. 'Normal' uses a static running-average. 'Confidence' uses a running average that changes dynamically based upon the confidence factor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setGyroRange (TSS_ID id, unsigned char range)

  Sets the measurment range used by the gyroscope sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setCompassRange (TSS_ID id, unsigned char range)

  Sets the measurement range used by the compass sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getTareOrientQuat (TSS_ID id, TSS_Quaternion *quat)

  Reads the tare orientation as a quaternion. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getTareOrientQuatf4 (TSS_ID id, float *quat)

  Reads the tare orientation as a quaternion. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getTareOrientMat (TSS_ID id, TSS_Matrix *mat)

  Reads the tare orientation as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getTareOrientMatf9 (TSS_ID id, float *mat)

  Reads the tare orientation as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getRhoDataAccelerometer (TSS_ID id, unsigned char *mode, float *min_static, float *max)

  Reads the rho mode and rho data for the accelerometer. Rho mode, min/static rho, and the max rho (if the rho mode is 'confidence') are returned. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getRhoDataCompass (TSS_ID id, unsigned char *mode, float *min_static, float *max)

  Reads the rho mode and rho data for the compass. Rho mode, min/static rho, and the max rho (if the rho mode is 'confidence') are returned. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getActualUpdateRate (TSS_ID id, unsigned int *rate)

  Reads the amount of time the last frame took. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getRefVecCompass (TSS_ID id, TSS_Vector *vec)

  Reads the single mode compass reference vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getRefVecCompassf3 (TSS_ID id, float *vec)

  Reads the single mode compass reference vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getRefVecAccelerometer (TSS_ID id, TSS_Vector *vec)

  Reads the single mode accelerometer reference vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getRefVecAccelerometerf3 (TSS_ID id, float *vec)

  Reads the single mode accelerometer reference vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getRefVecMode (TSS_ID id, unsigned char *mode)

  Reads the reference vector mode. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getMultiRefVecCompass (TSS_ID id, unsigned char idx, TSS_Vector *vec)

Reads the multi mode compass reference vector at index. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error getMultiRefVecCompassf3 (TSS_ID id, unsigned char idx, float *vec)

  Reads the multi mode compass reference vector at index. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getMultiRefChkVecCompass (TSS_ID id, unsigned char idx, TSS_Vector *vec)

  Reads the multi mode compass reference check vector at index. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getMultiRefChkVecCompassf3 (TSS_ID id, unsigned char idx, float *vec)

  Reads the multi mode compass reference check vector at index. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getMultiRefVecAccelerometer (TSS_ID id, unsigned char idx, TSS_Vector *vec)

  Reads the multi mode accelerometer reference vector at index. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getMultiRefVecAccelerometerf3 (TSS_ID id, unsigned char idx, float *vec)

  Reads the multi mode accelerometer reference vector at index. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getMultiRefChkVecAccelerometer (TSS_ID id, unsigned char idx, TSS_Vector *vec)

  Reads the multi mode accelerometer reference check vector at index. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getMultiRefChkVecAccelerometerf3 (TSS_ID id, unsigned char idx, float *vec)

  Reads the multi mode accelerometer reference check vector at index. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getGyroEnabled (TSS_ID id, boolean *is_enabled)

  Reads the enabled state of the gyros. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAccelerometerEnabled (TSS_ID id, boolean *is_enabled)

  Reads the enabled state of the Accelerometer. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getCompassEnabled (TSS_ID id, boolean *is_enabled)

  Reads the enabled state of the Compass. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAxisDirections (TSS_ID id, AxisDirections *axis_dir)

  Reads the current axis direction configuration of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getOversampleRate (TSS_ID id, unsigned char *rate)

  Read oversample rate. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAvgPercent (TSS_ID id, float *percent)

  Reads what percentage of running average to use on the sensor's orientation. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getDesiredUpdateRate (TSS_ID id, unsigned int *rate)

  Reads the current desired update rate. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getKalmanMat (TSS_ID id, TSS_Matrix *mat)

  Read Kalman filter's covariance 3x3 matrix. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getKalmanMatf9 (TSS_ID id, float *mat)

  Read Kalman filter's covariance 3x3 matrix. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAccelerometerRange (TSS_ID id, unsigned char *range)

  Read accelerometer sensitivity range. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getMultiRefWeightPwr (TSS_ID id, float *pow)

  Get the weighting power for multi reference vector weights. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getMultiRefResolution (TSS_ID id, unsigned char *num_division, unsigned char *num_vecs)

  Reads number of cell divisions and number of nearby vectors per cell for the multi reference lookup table. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getNumMultiRefCells (TSS_ID id, unsigned int *num_cells)

  Read number of multi reference cells. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getFilterMode (TSS_ID id, boolean *is_enabled)

  Read whether the Kalman Filter is enabled. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error getRunningAvgMode (TSS_ID id, unsigned char *mode)

  Reads the mode that the running-average method uses. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getGyroRange (TSS_ID id, unsigned char *range)

  Reads the current operation range of the 3-Space Sensor's gyroscope. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getCompassRange (TSS_ID id, unsigned char *range)

  Reads the current operation range of the 3-Space Sensor's compass. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setCompassCalibrationParam (TSS_ID id, TSS_Matrix mat, TSS_Vector bias)

  Sets the compass calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setCompassCalibrationParamf9f3 (TSS_ID id, float *mat, float *bias)

  Sets the compass calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setAccelerometerCalibrationParam (TSS_ID id, TSS_Matrix mat, TSS_Vector bias)

  Sets the accelerometer calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setAccelerometerCalibrationParamf9f3 (TSS_ID id, float *mat, float *bias)

  Sets the accelerometer calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getCompassCalibrationParam (TSS_ID id, TSS_Matrix *mat, TSS_Vector *bias)

  Gets the compass calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getCompassCalibrationParamf9f3 (TSS_ID id, float *mat, float *bias)

  Gets the compass calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAccelerometerCalibrationParam (TSS_ID id, TSS_Matrix *mat, TSS_Vector *bias)

  Gets the accelerometer calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getAccelerometerCalibrationParamf9f3 (TSS_ID id, float *mat, float *bias)

  Gets the accelerometer calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getGyroCalibrationParam (TSS_ID id, TSS_Matrix *mat, TSS_Vector *bias)

  Gets the gyro calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getGyroCalibrationParamf9f3 (TSS_ID id, float *mat, float *bias)

  Gets the gyro calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error calibrateGyro (TSS_ID id)

  Puts the sensor in gyro calibration mode. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setGyroCalibrationParam (TSS_ID id, TSS_Matrix mat, TSS_Vector bias)

  Sets the gyro calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setGyroCalibrationParamf9f3 (TSS_ID id, float *mat, float *bias)

  Sets the gyro calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setLookupTblVertVal (TSS_ID id, unsigned char type, unsigned short idx, TSS_Vector val)

  Set a vector entry in a 3-Space Sensor's vertex look-up table. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setLookupTblVertValf3 (TSS_ID id, unsigned char type, unsigned short idx, float *val)

  Set a vector entry in a 3-Space Sensor's vertex look-up table. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getLookupTblVertVal (TSS_ID id, unsigned char type, unsigned short idx, TSS_Vector *val)

  Reads a vector entry in a 3-Space Sensor's vertex look-up table. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getLookupTblVertValf3 (TSS_ID id, unsigned char type, unsigned short idx, float *val)

  Reads a vector entry in a 3-Space Sensor's vertex look-up table. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getPanID (TSS_ID id, unsigned short *pan_id)

Reads the Pan ID for the 3-Space Device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.

- EXPORT TSS_Error setPanID (TSS_ID id, unsigned short pan_id)

  Sets the Pan ID for the 3-Space Device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.
- EXPORT TSS_Error getChannel (TSS_ID id, unsigned char *channel)

  Reads the wireless channel for the 3-Space Device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.
- EXPORT TSS_Error setChannel (TSS_ID id, unsigned char channel)

  Sets the wireless channel for the 3-Space Device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.
- EXPORT TSS_Error setLEDMode (TSS_ID id, unsigned char mode)

  Sets the mode of the device's LED. This call will error if the inputted id is not for a 3-Space Dongle, 3-Space Wireless Sensor, or a 3-Space Bluetooth Sensor.
- EXPORT TSS_Error commitWirelessSettings (TSS_ID id)

  Commits wireless configuration settings to the device's non-volitile memory. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.
- EXPORT TSS_Error getWirelessAddress (TSS_ID id, unsigned short *address)

  Reads the wireless address of the device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.
- EXPORT TSS_Error setWirelessAddress (TSS_ID id, unsigned short address)

  Sets the wireless address of the device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.
- EXPORT TSS_Error getLEDMode (TSS_ID id, unsigned char *mode)

  Reads the mode of the device's LED. This call will error if the inputted id is not for a 3-Space Dongle, 3-Space Wireless Sensor, or a 3-Space Bluetooth Sensor.
- EXPORT TSS_Error getBatteryVoltage (TSS_ID id, float *bat_volt)

  Reads the current battery voltage for a 3-Space Sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor, 3-Space Data-Logging Sensor, or a 3-Space Bluetooth Sensor.
- EXPORT TSS_Error getBatteryLife (TSS_ID id, unsigned short *bat_life)

  Reads the current battery life left for a 3-Space Sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor, 3-Space Data-Logging Sensor, or a 3-Space Bluetooth Sensor.
- EXPORT TSS_Error getBatteryStatus (TSS_ID id, unsigned char *bat_status)

  Reads a status number indicating the state of the 3-Space Sensor's battery. This call will error if the inputted id is not for a 3-Space Wireless Sensor, 3-Space Data-Logging Sensor, or a 3-Space Bluetooth Sensor.
- EXPORT TSS_Error getWirelessHWID (TSS_ID id, unsigned char idx, unsigned int *hw_id)

  Reads the hardware ID (sensor serial number) that is associated with the logical ID given by an index. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error setWirelessHWID (TSS_ID id, unsigned char idx, unsigned int hw_id)

  Sets the logical ID given by an index to the hardware ID (sensor serial number) given by hw_id. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error getAllChannelNoise (TSS_ID id, unsigned char *channel_noise)

  Returns a char for each channel representing the level of noise on each respective channel. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error setWirelessRetries (TSS_ID id, unsigned char retries)

  Set the number of times the dongle will retry a failed transmission to a wireless sensor. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error getWirelessRetries (TSS_ID id, unsigned char *retries)

  Reads the number of times the dongle will retry a failed transmission to a wireless sensor. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error getWirelessSlotsOpen (TSS_ID id, unsigned char *slots)

  Reads how many more commands slots are available in the dongle's wireless send queue. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error getReceptionStrength (TSS_ID id, float *recep_strg)

Reads the relative strength of the most recent packet received by the dongle. This call will error if the inputted id is not for a 3-Space Dongle.

- EXPORT TSS_Error setHIDUpdateRate (TSS_ID id, unsigned char rate_ms)

  Sets the update rate of the mouse and joystick HID devices. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error getHIDUpdateRate (TSS_ID id, unsigned char *rate_ms)

  Reads the update rate of the mouse and joystick HID devices. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error getSoftwareVersion (TSS_ID id, char *version)

  Reads the version of the currenly installed firmware on the 3-Space device.
- EXPORT TSS_Error restoreFactorySettings (TSS_ID id)

  Restores all settings to factory settings.
- EXPORT TSS_Error commitSettings (TSS_ID id)

  Commits settings to non-volatile memory.
- EXPORT TSS_Error softwareReset (TSS_ID id)

  Resets the device without powering it down.
- EXPORT TSS_Error enableWatchdogTimer (TSS_ID id, unsigned int timeout)

  Enables the watchdog timer with the given timeout rate.
- EXPORT TSS_Error disableWatchdogTimer (TSS_ID id)

  Disables the watchdog timer.
- EXPORT TSS_Error enterFirmwareUpdateMode (TSS_ID id)

  Puts the sensor into firmware update mode.
- EXPORT TSS_Error getHardwareVersion (TSS_ID id, char *version)

  Reads the version and type of hardware being used on the 3-Space device.
- EXPORT TSS_Error setUARTRate (TSS_ID id, unsigned int rate)

  Sets the baud rate of the physical UART. This call will error if the inputted id is not for a 3-Space USB Sensor or a 3-Space Embedded Sensor.
- EXPORT TSS_Error getUARTRate (TSS_ID id, unsigned int *rate)

  Reads the baud rate of the physical UART. This call will error if the inputted id is not for a 3-Space USB Sensor or a 3-Space Embedded Sensor.
- EXPORT TSS_Error setUSBMode (TSS_ID id, unsigned char mode)

  Sets the communication mode for USB.
- EXPORT TSS_Error getUSBMode (TSS_ID id, unsigned char *mode)

  Gets the communication mode for USB.
- EXPORT TSS_Error setClockSpeed (TSS_ID id, unsigned int speed)

  Sets the clock speed of the device's MCU to the given value.
- EXPORT TSS_Error getClockSpeed (TSS_ID id, unsigned int *speed)

  Reads the clock speed of the device's MCU.
- EXPORT TSS_Error getSerialNumber (TSS_ID id, unsigned int *serial)

  Reads the serial number of the sensor.
- EXPORT TSS_Error setLEDColor (TSS_ID id, TSS_Color color)

  Sets the color of the LED on the sensor to the given RGB color.
- EXPORT TSS_Error setLEDColorf3 (TSS_ID id, float *color)

  Sets the color of the LED on the sensor to the given RGB color.
- EXPORT TSS_Error getLEDColor (TSS_ID id, TSS_Color *color)

  Reads the color of the LED on the sensor.
- EXPORT TSS_Error getLEDColorf3 (TSS_ID id, float *color)

  Reads the color of the LED on the sensor.
- EXPORT TSS_Error setJoystickEnabled (TSS_ID id, boolean do_enabled)

  Turns the data feed to the joystick on and off. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setJoystickLogicalID (TSS_ID id, unsigned char logical_id)

Set the logical ID of the wireless unit will operate as the joystick or disables joystick data transmission. This call will error if the inputted id is not for a 3-Space Dongle.

- EXPORT TSS_Error setMouseEnabled (TSS_ID id, boolean do_enabled)

  Turns the data feed to the mouse on and off. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMouseLogicalID (TSS_ID id, unsigned char logical_id)

  Set the logical ID of the wireless unit will operate as the mouse or disables mouse data transmission. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error getJoystickEnabled (TSS_ID id, boolean *is_enabled)

  Reads the enabled status for the HID Joystick for a 3-Space device. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getJoystickLogicalID (TSS_ID id, unsigned char *logical_id)

  Reads the logical ID of the wireless unit will operate as the joystick. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error getMouseEnabled (TSS_ID id, boolean *is_enabled)

  Turns the data feed to the mouse on and off. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getMouseLogicalID (TSS_ID id, unsigned char *logical_id)

  Reads the logical ID of the wireless unit will operate as the mouse. This call will error if the inputted id is not for a 3-Space Dongle.
- EXPORT TSS_Error **setControlMode** (TSS_ID id, unsigned char ctrl_class, unsigned char ctrl_idx, unsigned char hdl_idx)
- EXPORT TSS_Error **setControlData** (TSS_ID id, unsigned char ctrl_class, unsigned char ctrl_idx, unsigned char data_idx, float data)
- EXPORT TSS_Error **getControlMode** (TSS_ID id, unsigned char ctrl_class, unsigned char ctrl_idx, unsigned char *hdl_idx)
- EXPORT TSS_Error **getControlData** (TSS_ID id, unsigned char ctrl_class, unsigned char ctrl_idx, unsigned char data_idx, float *data)
- EXPORT TSS_Error setButtonGyroDisableLength (TSS_ID id, unsigned char length)

  Sets how long, in frames, the gyros should be disabled after one of the physical buttons on the sensor is pressed. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getButtonGyroDisableLength (TSS_ID id, unsigned char *length)

  Reads how long, in frames, the gyros are disabled after one of the physical buttons on the sensor is pressed. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error getButtonState (TSS_ID id, ButtonState *cur_state)

  Reads the current state of the sensor's physical buttons. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setMouseRelative (TSS_ID id, boolean do_relative)

  Puts the emulated HID mouse in absolute or relative mode.
- EXPORT TSS_Error getMouseRelative (TSS_ID id, boolean *is_relative)

  Reads the current emulated HID mouse absolute/relative state.
- EXPORT TSS_Error setJoystickMousePresent (TSS_ID id, boolean do_joy_present, boolean do_mouse_present)

  Sets whether the joystick and mouse are present or removed.
- EXPORT TSS_Error getJoystickMousePresent (TSS_ID id, boolean *is_joy_present, boolean *is_mouse_present)

  Reads whether the joystick and mouse are present or removed.
- EXPORT TSS_Error setGlobalAxis (TSS_ID id, char *joy_or_mouse, char *device_axis, TSS_Vector local_axis, TSS_Vector global_axis, float deadzone, float scale, float power)

  Sets an axis of the desired emulated input device as a 'Global Axis' style axis. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setScreenPointAxis (TSS_ID id, char *joy_or_mouse, char *device_axis, float dist_from_screen, float dist_on_axis, char *collision_component, char *sensor_dir, float button_halt)

  Sets an axis of the desired emulated input device as a 'Screen Point Axis' style axis. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error disableAxis (TSS_ID id, char *joy_or_mouse, char *device_axis)

  Disables an axis on the passed in device. This call will error if the inputted id is for a 3-Space Dongle.

- EXPORT TSS_Error setPhysicalButton (TSS_ID id, char *joy_or_mouse, unsigned char but_idx, char *button_bind)

  Binds a sensor's physical button to an emulated device's button. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setShakeButton (TSS_ID id, char *joy_or_mouse, unsigned char but_idx, float threshold)

  Sets up an emulated device's button such that it is 'pressed' when the sensor is shaken. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error setOrientationButton (TSS_ID id, char *joy_or_mouse, unsigned char but_idx, TSS_Vector local_axis, TSS_Vector global_axis, float max_dist)

  Sets up a device's button such that it is 'pressed' when a reference vector aligns itself with a local vector. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error disableButton (TSS_ID id, char *joy_or_mouse, unsigned char but_idx)

  Disables a button on the passed in emulated device. This call will error if the inputted id is for a 3-Space Dongle.
- EXPORT TSS_Error stopAsynchronous (TSS_ID id)

  Stops all asynchronous transmissions from the wireless sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error updateAsynchData ()

  Updates the internaly stored asynchronous data gathered from the connected 3-Space Dongles.
- EXPORT TSS_Error getAsynchDataFloat (TSS_ID id, float *data, int *timestamp, int *required_size)

  Retrieves asynchronous data for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error getAsynchDataUInt (TSS_ID id, unsigned int *data, int *timestamp, int *required_size)

  Retrieves asynchronous data for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error getAsynchDataUShort (TSS_ID id, unsigned short *data, int *timestamp, int *required_size)

  Retrieves asynchronous data for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error getAsynchDataUChar (TSS_ID id, unsigned char *data, int *timestamp, int *required_size)

  Retrieves asynchronous data for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltTaredOrientQuat (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltTaredOrientEuler (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltTaredOrientMatrix (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltTaredOrientAxisAngle (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltTaredOrientFwdDwn (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

- EXPORT TSS_Error asynchGetFiltOrientQuat (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltOrientEuler (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltOrientMat (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltOrientAxisAngle (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltOrientFwdDwn (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltTaredFwdDwnVecSensFrame (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetFiltNorthEarthVecSensFrame (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetAllSensorsNormalized (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetGyroNormalized (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetAccelerometerNormalized (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetCompassNormalized (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetAccelerometerUnfiltered (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetCompassUnfiltered (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetAllSensorsRaw (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

- EXPORT TSS_Error asynchGetGyroRaw (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetAccelerometerRaw (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetCompassRaw (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.
- EXPORT TSS_Error asynchGetButtonState (TSS_ID id, unsigned short interval, unsigned short duration)

  Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

4.1.1 Detailed Description

4.1.2 Typedef Documentation

4.1.2.1 typedef enum Axis_Order Axis_Order

An enum expressing the different possible configurations of axis assignment between the axes of rotation the physical 3-Space sensor experiences (*Natural Axes*) and the axes of rotation depiced by the sensor's output data (*Output Axes*).

When dealing with axis directions, we must first define two concepts, *Natural Axes* and *Output Axes*. *Natural Axes* are the axes of rotation about which the physical 3-Space sensor experiences changes in orientation. *Output Axes* are the axes of rotation about which data received from a 3-Space sensor depicts. This distinction is necessary because, for instance, it is possible to use the axis directions setting to express rotations about the physical 3--Space Sensor's X-Axis (the *Natural* X-Axis) as rotations about the Y-Axis in the sensor's output data (the *Output* Y-Axis).

When interpreting the axis directions, a combination of the three letter "X", "Y", and "Z" are used. Depending on the order of arrangement of the three letter, a different mapping of *Natural Axes* to *Output Axes* is expressed. The position of the given letter in the three letter set expresses the *Natural Axis* and the letter residing at that position expresses the *Output Axis*. For instance, an axis directions setting of "YZX" indicates that the *Natural* X-Axis is being mapped the *Output* Y-Axis, the *Natural* Y-Axis is being mapped to the *Output* Z-Axis, and the *Natural* Z-Axis is being mapped to the *Output* X-Axis. This means that data dealing with rotations experienced by the sensor hardware on its X-Axis are being mapped to the Y-Axis of the output data, data dealing with rotations experienced by the sensor hardware on its Y-Axis are being mapped to the Z-Axis of the output data, and data dealing with rotations experienced by the sensor hardware on its Z-Axis are being mapped to the X-Axis of the output data.

4.1.2.2 typedef struct ButtonState ButtonState

Stores the state of the physical buttons on a 3-Space sensor.

Designation of "left" and "right" are based on the assumed that the sensor' side with physical buttons is facing towards the point of reference with the LED on that side pointed upward.

4.1.2.3 typedef struct SensorData SensorData

A storage structure for a dump of all data from a 3-Space Sensor's on-board sensors.

The three on-board sensors used on a 3-Space Sensor are a gyroscope, and accelerometer, and a compass. The gyroscope expresses rates of rotations about each of its major axes. The accelerometer expresses any experienced accleration about each of its major axes. The compass expresses a three dimensional vector pointed towards magnetic north.

4.1.2.4 typedef struct TSS_AxisAngle TSS_AxisAngle

A storage structure for orientations expressed as an Axis/Angle pair.

The stored orientation holds a normalized three dimensional vector and an angle in radians.

4.1.2.5 typedef struct TSS_Euler TSS_Euler

A storage structure for orientations expressed as rotations about the three cartesian axes.

TSS_Euler values returned from API calls always return with the assumed rotation order X (Pitch), Y (Yaw), Z (Roll).

4.1.2.6 typedef struct TSS_Matrix TSS_Matrix

A storage structure for orientations expressed as a 3x3 TSS_Matrix.

Matricies returned from API calls are always in row-major. Fields for the structure are named based on the following template: m[ROW][COLUMN]. This means that matrix field m12 corresponds to the value in row 1 column 2.

4.1.3 Enumeration Type Documentation

4.1.3.1 enum Axis_Order

An enum expressing the different possible configurations of axis assignment between the axes of rotation the physical 3-Space sensor experiences (*Natural Axes*) and the axes of rotation depiced by the sensor's output data (*Output Axes*).

When dealing with axis directions, we must first define two concepts, *Natural Axes* and *Output Axes*. *Natural Axes* are the axes of rotation about which the physical 3-Space sensor experiences changes in orientation. *Output Axes* are the axes of rotation about which data received from a 3-Space sensor depicts. This distinction is necessary because, for instance, it is possible to use the axis directions setting to express rotations about the physical 3--Space Sensor's X-Axis (the *Natural* X-Axis) as rotations about the Y-Axis in the sensor's output data (the *Output* Y-Axis).

When interpreting the axis directions, a combination of the three letter "X", "Y", and "Z" are used. Depending on the order of arrangement of the three letter, a different mapping of *Natural Axes* to *Output Axes* is expressed. The position of the given letter in the three letter set expresses the *Natural Axis* and the letter residing at that position expresses the *Output Axis*. For instance, an axis directions setting of "YZX" indicates that the *Natural* X-Axis is being mapped the *Output* Y-Axis, the *Natural* Y-Axis is being mapped to the *Output* Z-Axis, and the *Natural* Z-Axis is being mapped to the *Output* X-Axis. This means that data dealing with rotations experienced by the sensor hardware on its X-Axis are being mapped to the Y-Axis of the output data, data dealing with rotations experienced by the sensor hardware on its Y-Axis are being mapped to the Z-Axis of the output data, and data dealing with rotations experienced by the sensor hardware on its Z-Axis are being mapped to the X-Axis of the output data.

Enumerator:

XYZ Map the *Natural* X-Axis to the *Output* X-Axis, *Natural* Y-Axis to the *Output* Y-Axis, and *Natural* Z-Axis to the *Output* Z-Axis.

XZY Map the *Natural* X-Axis to the *Output* X-Axis, *Natural* Y-Axis to the *Output* Z-Axis, and *Natural* Z-Axis to the *Output* Y-Axis.

YXZ Map the *Natural* X-Axis to the *Output* Y-Axis, *Natural* Y-Axis to the *Output* X-Axis, and *Natural* Z-Axis to the *Output* Z-Axis.

YZX Map the *Natural* X-Axis to the *Output* Y-Axis, *Natural* Y-Axis to the *Output* Z-Axis, and *Natural* Z-Axis to the *Output* X-Axis.

ZXY Map the *Natural* X-Axis to the *Output* Z-Axis, *Natural* Y-Axis to the *Output* X-Axis, and *Natural* Z-Axis to the *Output* Y-Axis.

ZYX Map the *Natural* X-Axis to the *Output* Z-Axis, *Natural* Y-Axis to the *Output* Y-Axis, and *Natural* Z-Axis to the *Output* X-Axis.

4.1.3.2 enum TSS_Error

An enum expressing the different types of errors a 3-Space API call can return.

Enumerator:

TSS_NO_ERROR The API call successfuly executed.

TSS_INVALID_COMMAND The API call was made on a device type that does not suppport the attemped command.

TSS_INVALID_ID The TSS_ID parameter passed in to an API call is not associated with a connected 3-Space device.

TSS_INVALID_IDX The index passed into a Wireless Sensor API call is used to look up a sensor serial number for a given dongle. If either the index is not 0<= index < 15 (the valid range of the index) or the stored serial number is 0 (meaning no wireless sensor is assigned to the index) this error is returned.

TSS_WARNING_OLD_FIRMWARE The API call executed had to fall back to an alternate implimentation to execute because the 3-Space device being used is on an old build of firmware. Firmware updating is suggested if this error is returned.

TSS_ERROR_OLD_FIRMWARE The API call executed could not be completed because the connected 3--Space device has firmware that is too old to support the call. Firmware updating is suggested if this error is returned.

TSS_ERROR_USB_CONNECTION When creating a 3-Space device that is connected over USB there was an issue communicating with the intended device.

TSS_ERROR_WIR_CONNECTION When creating a 3-Space device that is communicating through a 3--Space Dongle there was an issue communicating with the intended device.

TSS_ERROR_WRITE The API call executed failed to write all the data necisary to execute the command to the intended serial port.

TSS_ERROR_READ The API call executed failed to read all the data necisary to execute the command to the intended serial port.

TSS_ERROR_NO_SENSOR_FOUND no sensor found when looking for sensors

4.1.4 Function Documentation

4.1.4.1 EXPORT TSS_Error addSensorToDongle (TSS_ID *id*, unsigned char *idx*, unsigned int *hw_id*, AxisDirections * *axis_dir*, TSS_ID * *wl_id*)

Registers a 3-Space Wireless Sensor with a 3-Space Dongle and connects the wireless sensor.

Parameters

id	The identifier for the 3-Space Dongle.
idx	The index into the dongle's wireless table to register the wireless sensor to. May be between 0 and 15.
hw_id	The serial number of the wireless sensor to register.
axis_dir	An axis direction configuration to default the wireless sensor to. May be set to NULL if no default axis direction is desired.
wl_id	The unique identifier for the wireless sensor will be written to the variable referenced by this pointer.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.2 EXPORT TSS_Error asynchGetAccelerometerNormalized (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current normalized data being read from the accelerometer on the wireless sensor. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.3 EXPORT TSS_Error asynchGetAccelerometerRaw (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current raw data being read from the accelerometer on the wireless sensor. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.4 EXPORT TSS_Error asynchGetAccelerometerUnfiltered (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current unfiltered (but compensated) data being read from the accelerometer on the wireless sensor. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.5 EXPORT TSS_Error asynchGetAllSensorsNormalized (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current normalized data being read from the collection of on-board sensors on the wireless sensor. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (Gyro_x, Gyro_y, Gyro_z, Accelerometer_x, Accelerometer_y, Accelerometer_z, Compass_x, Compass_y, Compass_z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.6 EXPORT TSS_Error asynchGetAllSensorsRaw (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current raw data being read from the collection of on-board sensors on the wireless sensor. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (Gyro_x, Gyro_y, Gyro_z, Accelerometer_x, Accelerometer_y, Accelerometer_z, Compass_x, Compass_y, Compass_z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.7 EXPORT TSS_Error asynchGetButtonState (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current state of the sensor's physical buttons. When retrieving the data, the stored data is casted as an unsigned character. The unsigned character value represents the button states as such: (1: right buton down, 2: left button down, 3: both buttons down)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.8 EXPORT TSS_Error asynchGetCompassNormalized (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current normalized data being read from the compass on the wireless sensor. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.9 EXPORT TSS_Error asynchGetCompassRaw (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current raw data being read from the compass on the wireless sensor. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.10 EXPORT TSS_Error asynchGetCompassUnfiltered (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current unfiltered (but compensated) data being read from the compass on the wireless sensor. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.11 EXPORT TSS_Error asynchGetFiltNorthEarthVecSensFrame (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current North and Earth vectors determined by the Filtered Tared Orientation of the sensor. The data is returned such that the North and Earth vectors' coordinate system is defined by the sensor reference frame. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (North_x, North_y, North_z, Earth_x, Earth_y, Earth_z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.12 EXPORT TSS_Error asynchGetFiltOrientAxisAngle (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Untared Orientation of the sensor as an axis angle construct. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z, angle) with the angle being in radians

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.13 EXPORT TSS_Error asynchGetFiltOrientEuler (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Untared Orientation of the sensor as a set of euler angles. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (pitch, yaw, roll) in radians

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.14 EXPORT TSS_Error asynchGetFiltOrientFwdDwn (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Untared Orientation of the sensor as a pair of two vectors (A Forward vector and a Down vector). When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (Forward_x, Forward_y, Forward_z, Down_x, Down_y, Down_z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.15 EXPORT TSS_Error asynchGetFiltOrientMat (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Untared Orientation of the sensor as a 3x3 matrix. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted in row-major order.

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.16 EXPORT TSS_Error asynchGetFiltOrientQuat (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Untared Orientation of the sensor as a quaternion. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z, w)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.17 EXPORT TSS_Error asynchGetFiltTaredFwdDwnVecSensFrame (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Tared Orientation of the sensor as a pair of two vectors (A Forward vector and a Down vector). This version returns the data as the Forward and Down vectors of the coordinate system defined by the sensor reference frame. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (Forward_x, Forward_y, Forward_z, Down_x, Down_y, Down_z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.18 EXPORT TSS_Error asynchGetFiltTaredOrientAxisAngle (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Tared Orientation of the sensor as an axis angle construct. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z, angle) with the angle being in radians

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.19 EXPORT TSS_Error asynchGetFiltTaredOrientEuler (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Tared Orientation of the sensor as a set of euler angles. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (pitch, yaw, roll) in radians

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.20 EXPORT TSS_Error asynchGetFiltTaredOrientFwdDwn (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Tared Orientation of the sensor as a pair of two vectors (A Forward vector and a Down vector). When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (Forward_x, Forward_y, Forward_z, Down_x, Down_y, Down_z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.21 EXPORT TSS_Error asynchGetFiltTaredOrientMatrix (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Tared Orientation of the sensor as a 3x3 matrix. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted in row-major order.

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.22 EXPORT TSS_Error asynchGetFiltTaredOrientQuat (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current Filtered Tared Orientation of the sensor as a quaternion. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z, w)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.23 EXPORT TSS_Error asynchGetGyroNormalized (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current normalized data being read from the gyroscope on the wireless sensor. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.24 EXPORT TSS_Error asynchGetGyroRaw (TSS_ID *id,* unsigned short *interval,* unsigned short *duration*)

Begins the asynchronous session for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

This session will be transmitting the current raw data being read from the gyroscope on the wireless sensor. When retrieving the data, the stored data is casted as an array of floats. The array will be formatted as such: (x, y, z)

Parameters

id	The identifier for the 3-Space device.
interval	An integer whose value expresses how long, in milliseconds, the wireless sensor should wait before attempting to transmit an asynchronous packet.
duration	An integer whose value expresses how long, in milliseconds, the wireless sensor should transmit asynchronous packets. Once the elapsed time since the start of the asynchronous session exceeds duration, the sensor will stop transmitting asynchronous packets on its own. If duration is 0xffff (65535 in decimal), the sensor will tranmit asynchronous packets until either the sensor is powered down or another asynchronous command is sent with a finite duration value. A duration of 0 will effectively stop all asynchronous transmissions from the wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.25 EXPORT TSS_Error calibrateGyro (TSS_ID *id*)

Puts the sensor in gyro calibration mode. This call will error if the inputted id is for a 3-Space Dongle.

It will collect a few frames of data from the gyro to determine its bias. It will return to normal operation after this or if the sensor is reset.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.26 EXPORT TSS_Error closeTSDevice (TSS_ID *id*)

Disconnects the 3-Space device associated with the inputted ID. If connected through a serial port, that port is closed.

Parameters

id	The identifier for the device to disconnect.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.27 EXPORT TSS_Error commitSettings (TSS_ID *id*)

Commits settings to non-volatile memory.

This will cause them to persist even if the sensor is reset.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.28 EXPORT TSS_Error commitWirelessSettings (TSS_ID *id*)

Commits wireless configuration settings to the device's non-volitile memory. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.

The settings committed with this command are:

- PanID
- Wireless Channel
- Wireless Address

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.29 EXPORT TSS_ID createTSDevice (const ComPort *com_port*)

Given a serial port, the type of 3-Space device is determined from the passed in structure and the device is connected to. An identifier for the device is returned.

Parameters

com_port	The serial port to connect the device on.

Returns

An identifier for the created device.

4.1.4.30 EXPORT TSS_ID createTSDeviceStr (const char * *com_port,* TSS_Type *type*)

Given a serial port and the type of 3-Space device, the device is connected to. An identifier for the device is returned.

Parameters

com_port	The serial port name.
type	The type of 3-Space device connected on the serial port.

Returns

An identifier for the created device.

4.1.4.31 EXPORT TSS_Error disableAxis (TSS_ID *id*, char * *joy_or_mouse*, char * *device_axis*)

Disables an axis on the passed in device. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
joy_or_mouse	A null-terminated string whose value may be either 'mouse' or 'joystick'.
device_axis	A null-terminated string whose value may be either 'X' or 'Y'. This string defines what axis of the device is to be configured.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.32 EXPORT TSS_Error disableButton (TSS_ID *id*, char * *joy_or_mouse*, unsigned char *but_idx*)

Disables a button on the passed in emulated device. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
joy_or_mouse	A null-terminated string whose value may be either 'mouse' or 'joystick'.
but_idx	The value that defines which button on the emulated device to configure. This value is in the range 0 - 7.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.33 EXPORT TSS_Error disableMassStorageMode (TSS_ID *id*)

When disabled, the 3-Space Sensor's mass storage features are hidden from the host machine. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.34 EXPORT TSS_Error disableWatchdogTimer (TSS_ID *id*)

Disables the watchdog timer.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.35 EXPORT TSS_Error enableMassStorageMode (TSS_ID *id*)

When enabled, the 3-Space Sensor acts as a mass storage device on the host system. This is useful for reading/writing log files to the sensor. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.36 EXPORT TSS_Error enableWatchdogTimer (TSS_ID *id,* unsigned int *timeout*)

Enables the watchdog timer with the given timeout rate.

If a frame takes more than this amount of time, the sensor will automatically reset. This is useful for dealing with sensor hangs or crashes, as the sensor would reset and continue normal operation.

Parameters

id	The identifier for the 3-Space device.
timeout	The length of time in microseconds to wait before timing out when the watchdog timer is active.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.37 EXPORT TSS_Error enterFirmwareUpdateMode (TSS_ID *id*)

Puts the sensor into firmware update mode.

This will cease normal operation until the firmware update mode is instructed to return the sensor to normal operation.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.38 EXPORT TSS_Error finishWrite (TSS_ID *id*)

Tells the 3-Space device to stop writting. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.39 EXPORT TSS_Error formatInitSDCard (TSS_ID *id*)

The SD card contained within the 3-Space Sensor is formatted to the FAT-32 file system and initialized for writing/reading. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.40 EXPORT TSS_Error getAccelerometerCalibrationParam (TSS_ID *id,* TSS_Matrix * *mat,* TSS_Vector * *bias*)

Gets the accelerometer calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by is written to the referenced variable.
bias	The bias of the calibration is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.41 EXPORT TSS_Error getAccelerometerCalibrationParamf9f3 (TSS_ID *id,* float * *mat,* float * *bias*)

Gets the accelerometer calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by is written to the referenced array of nine floats. The array is arranged in row-major order.
bias	The bias of the calibration is written to the referenced array of three floats. The array is arranged as: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.42 EXPORT TSS_Error getAccelerometerEnabled (TSS_ID *id*, boolean * *is_enabled*)

Reads the enabled state of the Accelerometer. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
is_enabled	The state of the Accelerometer is written to the referenced variable. True if the Accelerometer is enabled. False if the Accelerometer is disabled.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.43 EXPORT TSS_Error getAccelerometerNormalized (TSS_ID *id*, TSS_Vector * *accelerometer*)

Returns the normalized accelerometer vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
accelerometer	The normalized data from the 3-Space Sensor's on-board accelerometer is written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.44 EXPORT TSS_Error getAccelerometerNormalizedf3 (TSS_ID *id*, float * *accelerometer*)

Returns the normalized accelerometer vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
accelerometer	The normalized data from the 3-Space Sensor's on-board accelerometer is written to the referenced array of 3 floats. The floats are ordered: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.45 EXPORT TSS_Error getAccelerometerRange (TSS_ID *id*, unsigned char * *range*)

Read accelerometer sensitivity range. This call will error if the inputted id is for a 3-Space Dongle.

Higher ranges can detect and report larger accelerations, but are not as accurate for smaller ones.

Parameters

id	The identifier for the 3-Space device.
range	The intended range for the accelerometer. Expected values are: 0 - 2G. 16 - 4G. 48 - 8G.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.46 EXPORT TSS_Error getAccelerometerRaw (TSS_ID *id*, TSS_Vector * *accelerometer*)

Returns the unaltered data of the 3-Space Sensor's on-board accelerometer. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
accelerometer	The unaltered data from the 3-Space Sensor's on-board accelerometer is written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.47 EXPORT TSS_Error getAccelerometerRawf3 (TSS_ID *id*, float * *accelerometer*)

Returns the unaltered data of the 3-Space Sensor's on-board accelerometer. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
accelerometer	The unaltered data from the 3-Space Sensor's on-board accelerometer is written to the referenced array of three floats. The array is structured: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.48 EXPORT TSS_Error getAccelerometerUnfiltered (TSS_ID *id,* TSS_Vector * *accelerometer*)

Returns the unfiltered (but compensated) data of the 3-Space Sensor's on-board accelerometer. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
accelerometer	The data from the 3-Space Sensor's on-board accelerometer is written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.49 EXPORT TSS_Error getAccelerometerUnfilteredf3 (TSS_ID *id,* float * *accelerometer*)

Returns the unfiltered (but compensated) data of the 3-Space Sensor's on-board accelerometer. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
accelerometer	The data from the 3-Space Sensor's on-board accelerometer is written to the referenced array of 3 floats. The floats are ordered: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.50 EXPORT TSS_Error getActualUpdateRate (TSS_ID *id,* unsigned int * *rate*)

Reads the amount of time the last frame took. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
rate	The amount of time the last frame of the kalman filter took to process in microseconds is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.51 EXPORT TSS_Error getAllChannelNoise (TSS_ID *id,* unsigned char * *channel_noise*)

Returns a char for each channel representing the level of noise on each respective channel. This call will error if the inputted id is not for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
channel_noise	The noise of each wireless channel as an array of sixteen chars. Each of the chars' values range from 0...255 with 0 meaning the channel is completely clear, and 255 meaning that it is unusably noisy. The first element of the array corresponds to channel 11, and the last value corresponds to channel 26.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.52 EXPORT TSS_Error getAllSensorsNormalized (TSS_ID *id,* SensorData * *data*)

Returns the normalized data of the 3-Space Sensor's on-board sensors. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
data	The normalized data from the 3-Space Sensor's accelerometer, gyroscope, and compass are filled into the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.53 EXPORT TSS_Error getAllSensorsNormalizedf9 (TSS_ID *id,* float * *data*)

Returns the normalized data of the 3-Space Sensor's on-board sensors. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
data	The normalized data from the 3-Space Sensor's accelerometer, gyroscope, and compass are written to the referenced array of 9 floats. The floats are ordered gyro_x, gyro_y, gyro_z, accel_x, accel_y, accel_z, compass_x, compass_y, compass_z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.54 EXPORT TSS_Error getAllSensorsRaw (TSS_ID *id,* SensorData * *data*)

Returns the unaltered data of the 3-Space Sensor's on-board sensors. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
data	The unaltered data from the 3-Space Sensor's accelerometer, gyroscope, and compass are filled into the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.55 EXPORT TSS_Error getAllSensorsRawf9 (TSS_ID *id,* float * *data*)

Returns the unaltered data of the 3-Space Sensor's on-board sensors. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
data	The unaltered data from the 3-Space Sensor's accelerometer, gyroscope, and compass are written to the referenced array of 9 floats. The floats are ordered gyro_x, gyro_y, gyro_z, accel_x, accel_y, accel_z, compass_x, compass_y, compass_z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.56 EXPORT TSS_Error getAsynchDataFloat (TSS_ID *id,* float * *data,* int * *timestamp,* int * *required_size*)

Retrieves asynchronous data for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

When retrieving data, the stored data is casted as an array of floats.

Parameters

id	The identifier for the 3-Space device.
data	The asynchronous data is written to the referenced array of floats.
timestamp	The timestamp the 3-Space Dongle gave the asynchronous data is written to the referenced variable.
required_size	The required byte size that the referenced array needs to be.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.57 EXPORT TSS_Error getAsynchDataUChar (TSS_ID *id,* unsigned char * *data,* int * *timestamp,* int * *required_size*)

Retrieves asynchronous data for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

When retrieving data, the stored data is casted as an array of unsigned characters.

Parameters

id	The identifier for the 3-Space device.
data	The asynchronous data is written to the referenced array of unsigned characters.
timestamp	The timestamp the 3-Space Dongle gave the asynchronous data is written to the referenced variable.
required_size	The required byte size that the referenced array needs to be.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.58 EXPORT TSS_Error getAsynchDataUInt (TSS_ID *id,* unsigned int * *data,* int * *timestamp,* int * *required_size*)

Retrieves asynchronous data for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

When retrieving data, the stored data is casted as an array of unsigned integers.

Parameters

id	The identifier for the 3-Space device.
data	The asynchronous data is written to the referenced array of unsigned integers.
timestamp	The timestamp the 3-Space Dongle gave the asynchronous data is written to the referenced variable.
required_size	The required byte size that the referenced array needs to be.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.59 EXPORT TSS_Error getAsynchDataUShort (TSS_ID *id,* unsigned short * *data,* int * *timestamp,* int * *required_size*)

Retrieves asynchronous data for a 3-Space sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

When retrieving data, the stored data is casted as an array of unsigned short integers.

Parameters

id	The identifier for the 3-Space device.
data	The asynchronous data is written to the referenced array of unsigned short integers.
timestamp	The timestamp the 3-Space Dongle gave the asynchronous data is written to the referenced variable.
required_size	The required byte size that the referenced array needs to be.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.60 EXPORT void getAvailableComPorts (ComPort ** *tss_port_list,* ComPort ** *ukn_port_list,* unsigned int * *tss_list_size,* unsigned int * *ukn_list_size,* int *filter*)

Scans the host computer for all serial ports (virtual or physical). Ports returned are sorted into two lists based on whether the driver creating the port was a 3-Space driver.

Parameters

tss_port_list	A pointer to an array filled with serial ports created by 3-Space drivers. These ports have 3-Space hardware connected to them. This pointer will have the address of the array written to it, if not NULL.
ukn_port_list	A pointer to an array filled with serial ports not directly created by a 3-Space driver. These ports may have 3-Space hardware connected through RS-232 port. This pointer will have the address of the array written to it, if not NULL.
tss_list_size	A pointer to the number of listings written to the array pointed to by tss_port_list. This pointer is written to, if not NULL.
ukn_list_size	A pointer to the number of listings written to the array pointed to by ukn_port_list. This pointer is written to, if not NULL.
filter	A mask of what 3-Space device types were desired. Ports appended to tss_port_list will only have devices connected that match the mask.

4.1.4.61 EXPORT TSS_Error getAvgPercent (TSS_ID *id,* float * *percent*)

Reads what percentage of running average to use on the sensor's orientation. This call will error if the inputted id is for a 3-Space Dongle.

This is computed as follows:

total_orient = total_orient * percent

total_orient = total_orient + current_orient * (1 - percent)

current_orient = total_orient

If the percentage is 0, the running average will be shut off completely.

Parameters

id	The identifier for the 3-Space device.
percent	The percentage of the running average to use for the sensor's orientation. Expected values are within the range 0.0 to 1.0.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.62 EXPORT TSS_Error getAxisDirections (TSS_ID *id,* AxisDirections * *axis_dir*)

Reads the current axis direction configuration of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
axis_dir	The axis direction configuration is written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.63 EXPORT TSS_Error getBatteryLife (TSS_ID *id,* unsigned short * *bat_life*)

Reads the current battery life left for a 3-Space Sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor, 3-Space Data-Logging Sensor, or a 3-Space Bluetooth Sensor.

Parameters

id	The identifier for the 3-Space device.
bat_life	The percentage of the device's battery life remaining will be written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.64 EXPORT TSS_Error getBatteryStatus (TSS_ID *id,* unsigned char * *bat_status*)

Reads a status number indicating the state of the 3-Space Sensor's battery. This call will error if the inputted id is not for a 3-Space Wireless Sensor, 3-Space Data-Logging Sensor, or a 3-Space Bluetooth Sensor.

Parameters

id	The identifier for the 3-Space device.
bat_status	The current status of the battery is written to the refernced variable. Expected values are: • 1 represents fully charged. • 2 represents charging.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.65 EXPORT TSS_Error getBatteryVoltage (TSS_ID *id,* float * *bat_volt*)

Reads the current battery voltage for a 3-Space Sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor, 3-Space Data-Logging Sensor, or a 3-Space Bluetooth Sensor.

Parameters

id	The identifier for the 3-Space device.
bat_volt	The device's battery voltage will be written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.66 EXPORT TSS_Error getButtonGyroDisableLength (TSS_ID *id,* unsigned char * *length*)

Reads how long, in frames, the gyros are disabled after one of the physical buttons on the sensor is pressed. This call will error if the inputted id is for a 3-Space Dongle.

This setting helps to alleviate gyro disturbances caused by the buttons causing small shockwaves in the sensor.

Parameters

id	The identifier for the 3-Space device.
length	How many frames the gyros is disabled for when a button is pressed is written to the referenced variable. A setting of 0 means the gyros won't be disabled at all.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.67 EXPORT TSS_Error getButtonState (TSS_ID *id,* ButtonState * *cur_state*)

Reads the current state of the sensor's physical buttons. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
cur_state	The 3-Space device's button state is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.68 EXPORT TSS_Error getChannel (TSS_ID *id,* unsigned char * *channel*)

Reads the wireless channel for the 3-Space Device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.

Parameters

id	The identifier for the 3-Space device.
channel	The wireless channel of the 3-Space device is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.69 EXPORT TSS_Error getClock (TSS_ID *id,* TSS_Clock * *clock_data*)

Gets the current data and time of the 3-Space Sensor's internal clock. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.

Parameters

id	The identifier for the 3-Space device.
clock_data	The current time of the sensor's clock will be written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.70 EXPORT TSS_Error getClockc6 (TSS_ID *id,* unsigned char * *clock_data*)

Gets the current data and time of the 3-Space Sensor's internal clock. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor. This array will be formatted as: month, day, year, hour, minute, second.

Parameters

id	The identifier for the 3-Space device.
clock_data	The current time of the sensor's clock will be written to the referenced array of six chars.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.71 EXPORT TSS_Error getClockSpeed (TSS_ID *id,* unsigned int * *speed*)

Reads the clock speed of the device's MCU.

Parameters

id	The identifier for the 3-Space device.
speed	The clock speed of the MCU in Hz is written to the referenced variable. Expected values are: • 15000000 (15 MHz). • 30000000 (30 MHz). • 60000000 (60 MHz).

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.72 EXPORT TSS_Error getCompassCalibrationParam (TSS_ID *id,* TSS_Matrix * *mat,* TSS_Vector * *bias*)

Gets the compass calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by is written to the referenced variable.
bias	The bias of the calibration is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.73 EXPORT TSS_Error getCompassCalibrationParamf9f3 (TSS_ID *id,* float * *mat,* float * *bias*)

Gets the compass calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by is written to the referenced array of nine floats. The array is arranged in row-major order.
bias	The bias of the calibration is written to the referenced array of three floats. The array is arranged as: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.74 EXPORT TSS_Error getCompassEnabled (TSS_ID *id,* boolean * *is_enabled*)

Reads the enabled state of the Compass. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
is_enabled	The state of the Compass is written to the referenced variable. True if the Compass is enabled. False if the Compass is disabled.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.75 EXPORT TSS_Error getCompassNormalized (TSS_ID *id,* TSS_Vector * *compass*)

Returns the normalized compass vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
compass	The normalized data from the 3-Space Sensor's on-board compass is written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.76 EXPORT TSS_Error getCompassNormalizedf3 (TSS_ID *id*, float * *compass*)

Returns the normalized compass vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
compass	The normalized data from the 3-Space Sensor's on-board compass is written to the referenced array of 3 floats. The floats are ordered: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.77 EXPORT TSS_Error getCompassRange (TSS_ID *id*, unsigned char * *range*)

Reads the current operation range of the 3-Space Sensor's compass. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
range	The range mode of the compass is written to the referenced variable. Expected values are: 0 - +/- 0.88Ga mode 1 - +/- 1.30Ga mode 2 - +/- 1.90Ga mode 3 - +/- 2.50Ga mode 4 - +/- 4.00Ga mode 5 - +/- 4.70Ga mode 6 - +/- 5.60Ga mode 7 - +/- 8.10Ga mode

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.78 EXPORT TSS_Error getCompassRaw (TSS_ID *id*, TSS_Vector * *compass*)

Returns the unaltered compass vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
compass	The unaltered data from the 3-Space Sensor's on-board compass is written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.79 EXPORT TSS_Error getCompassRawf3 (TSS_ID *id,* float * *compass*)

Returns the unaltered compass vector of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
compass	The unaltered data from the 3-Space Sensor's on-board compass is written to the referenced array of 3 floats. The floats are ordered: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.80 EXPORT TSS_Error getCompassUnfiltered (TSS_ID *id,* TSS_Vector * *compass*)

Returns the unfiltered (but compensated) data of the 3-Space Sensor's on-board compass. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
compass	The data from the 3-Space Sensor's on-board compass is written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.81 EXPORT TSS_Error getCompassUnfilteredf3 (TSS_ID *id,* float * *compass*)

Returns the unfiltered (but compensated) data of the 3-Space Sensor's on-board compass. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
compass	The data from the 3-Space Sensor's on-board compass is written to the referenced array of 3 floats. The floats are ordered: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.82 EXPORT void getComPortInfo (const char * *com_port,* ComInfo * *com_info,* boolean *poll_device*)

Interrogates an indicated serial port for information about the device connected to it. Optionaly, the port may be polled to gain further information about the connected device. Polling a port may cause undesired effects if the device connected to the port is not a 3-Space device, as bytes will be written to the serial port. Its suggested that any polling done in an application be an optional action invoked by the user with proper warnings in place.

Parameters

com_port	The string name of the port to be interrogated.
com_info	Information gathered about the port will be written to this structure.
poll_device	If True, the port will have bytes written to it in order to gain information about the connected device. If False, only driver level information will be gathered.

4.1.4.83 EXPORT TSS_Error getConfidence (TSS_ID *id,* float * *confidence*)

Returns a value indicating how much the compass and accelerometer are to be trusted to be unit vectors pointing in the proper directions at the moment. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
confidence	The confidence value is written to the referenced variable. This value ranges from 0 to 1, with 1 being fully trusted and 0 being not trusted at all. This will change based on if the sensor is being moved and if it is near a strong magnetic field.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.84 EXPORT TSS_Error getDesiredUpdateRate (TSS_ID *id,* unsigned int * *rate*)

Reads the current desired update rate. This call will error if the inputted id is for a 3-Space Dongle.

If the filter takes less time to update during a given frame than this rate specifies, the frame will be padded out to take the desired amount of time. If the filter takes more time to update than this rate, this rate will be ignored.

Parameters

id	The identifier for the 3-Space device.
rate	The amount of time desired between each frame of kalman filter calculations in microseconds is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.85 EXPORT TSS_Error getDeviceType (TSS_ID *id,* TSS_Type * *device_type*)

Retrieves the type of 3-Space device connected as a string.

Parameters

id	The identifier for the 3-Space device.
device_type	An enum of TSS_Type that gives the devices base type specification, on wireless units can denote how it is connected.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.86 EXPORT TSS_Error getFilterMode (TSS_ID *id,* boolean * *is_enabled*)

Read whether the Kalman Filter is enabled. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
is_enabled	The Kalam Filter's state is written to the referenced variable. True if the filter is enabled. False if the filter is disabled.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.87 EXPORT TSS_Error getFiltGyroRate (TSS_ID *id,* TSS_Quaternion * *quat*)

Returns the gyro rates as determined by taking the difference between the current orientation and the previous one. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
quat	The quaternion representing this rate of change will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.88 EXPORT TSS_Error getFiltGyroRatef4 (TSS_ID *id,* float * *quat*)

Returns the gyro rates as determined by taking the difference between the current orientation and the previous one. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
quat	The quaternion representing this rate of change will be written to this array of four floats. The float are ordered x, y, z, w.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.89 EXPORT TSS_Error getFiltNorthEarthVecSensFrame (TSS_ID *id,* TSS_Vector * *north,* TSS_Vector * *earth*)

Returns the North and Earth vectors as determeined by the Kalman filter, relative to the global sensor references. Returned vectors are in the coordinate system defined by the sensor reference frame. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
earth	The earth (gravity) vector will be written to this structure.
north	The north (magnetic north) vector will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.90 EXPORT TSS_Error getFiltNorthEarthVecSensFramef3f3 (TSS_ID *id,* float * *north,* float * *earth*)

Returns the North and Earth vectors as determeined by the Kalman filter, relative to the global sensor references. Returned vectors are in the coordinate system defined by the sensor reference frame. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
earth	The earth (gravity) vector will be written to this array of 3 floats. The floats will be ordered x, y, z.
north	The north (magnetic north) vector will be written to this array of 3 floats. The floats will be ordered x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.91 EXPORT TSS_Error getFiltOrientAxisAngle (TSS_ID *id,* TSS_AxisAngle * *axis_ang*)

Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as an Axis Angle. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
axis_ang	The orientation will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.92 EXPORT TSS_Error getFiltOrientAxisAnglef4 (TSS_ID *id*, float * *axis_ang*)

Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as an Axis Angle. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
axis_ang	The orientation will be written to this array of four floats. The floats are written as x, y, z, angle. The x, y, and z values are elements of a normalized vector.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.93 EXPORT TSS_Error getFiltOrientEuler (TSS_ID *id*, TSS_Euler * *euler*)

Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a set of TSS_Euler angles. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
euler	The orientation will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.94 EXPORT TSS_Error getFiltOrientEulerf3 (TSS_ID *id*, float * *euler*)

Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a set of TSS_Euler angles. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
euler	The orientation will be written to this array of three floats. The order of elements will be x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.95 EXPORT TSS_Error getFiltOrientFwdDwn (TSS_ID *id*, TSS_Vector * *forward*, TSS_Vector * *down*)

Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a pair of two vectors (A Forward vector and a Down vector). This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
forward	The forward vector will be written to this structure.
down	The down vector will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.96 EXPORT TSS_Error getFiltOrientFwdDwnf3f3 (TSS_ID *id*, float * *forward*, float * *down*)

Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a pair of two vectors (A Forward vector and a Down vector). This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
forward	The forward vector will be written to this array of 3 floats. The floats will be ordered x, y, z.
down	The down vector will be written to this array of 3 floats. The floats will be ordered x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.97 EXPORT TSS_Error getFiltOrientMat (TSS_ID *id*, TSS_Matrix * *mat*)

Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mat	The orientation will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.98 EXPORT TSS_Error getFiltOrientMatf9 (TSS_ID *id*, float * *mat*)

Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mat	The orientation will be written to this array of nine floats. The floats will be written in row-major.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.99 EXPORT TSS_Error getFiltOrientQuat (TSS_ID *id*, TSS_Quaternion * *quat*)

Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a TSS_Quaternion. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
quat	The orientation will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.100 EXPORT TSS_Error getFiltOrientQuatf4 (TSS_ID *id*, float * *quat*)

Retrieves the current orientation of a 3-Space Sensor relative to its detected gravity and north vectors as a TSS_Quaternion. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
quat	The orientation will be written to this array as four floats. The order of elements will be x, y, z, w.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.101 EXPORT TSS_Error getFiltTaredFwdDwnVecSensFrame (TSS_ID *id*, TSS_Vector * *forward*, TSS_Vector * *down*)

Returns the forward and down vectors as determined by the Kalman filter, relative to the tare orientation. This version returns the data as the forward and down vectors of the coordinate system defined by the sensor reference frame. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
forward	The forward vector will be written to this structure.
down	The down vector will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.102 EXPORT TSS_Error getFiltTaredFwdDwnVecSensFramef3f3 (TSS_ID *id,* float * *forward,* float * *down*)

Returns the forward and down vectors as determined by the Kalman filter, relative to the tare orientation. This version returns the data as the forward and down vectors of the coordinate system defined by the sensor reference frame. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
forward	The forward vector will be written to this array of 3 floats. The floats will be ordered x, y, z.
down	The down vector will be written to this array of 3 floats. The floats will be ordered x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.103 EXPORT TSS_Error getFiltTaredOrientAxisAngle (TSS_ID *id,* TSS_AxisAngle * *axis_ang*)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as an Axis Angle. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
axis_ang	The orientation will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.104 EXPORT TSS_Error getFiltTaredOrientAxisAnglef4 (TSS_ID *id,* float * *axis_ang*)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as an Axis Angle. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
axis_ang	The orientation will be written to this array of four floats. The floats are written as x, y, z, angle. The x, y, and z values are elements of a normalized vector.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.105 EXPORT TSS_Error getFiltTaredOrientEuler (TSS_ID *id*, TSS_Euler * *euler*)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a set of TSS_Euler angles. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
euler	The orientation will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.106 EXPORT TSS_Error getFiltTaredOrientEulerf3 (TSS_ID *id*, float * *euler*)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a set of TSS_Euler angles. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
euler	The orientation will be written to this array of three floats. The order of elements will be x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.107 EXPORT TSS_Error getFiltTaredOrientFwdDwn (TSS_ID *id*, TSS_Vector * *forward*, TSS_Vector * *down*)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a pair of two vectors (A Forward vector and a Down vector). This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
forward	The forward vector will be written to this structure.
down	The down vector will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.108 EXPORT TSS_Error getFiltTaredOrientFwdDwnf3f3 (TSS_ID *id*, float * *forward*, float * *down*)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a pair of two vectors (A Forward vector and a Down vector). This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
forward	The forward vector will be written to this array of 3 floats. The floats will be ordered x, y, z.
down	The down vector will be written to this array of 3 floats. The floats will be ordered x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.109 EXPORT TSS_Error getFiltTaredOrientMat (TSS_ID *id*, TSS_Matrix * *mat*)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mat	The orientation will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.110 EXPORT TSS_Error getFiltTaredOrientMatf9 (TSS_ID *id*, float * *mat*)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mat	The orientation will be written to this array of nine floats. The floats will be written in row-major.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.111 EXPORT TSS_Error getFiltTaredOrientQuat (TSS_ID *id*, TSS_Quaternion * *quat*)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a TSS_Quaternion. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
quat	The orientation will be written to this structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.112 EXPORT TSS_Error getFiltTaredOrientQuatf4 (TSS_ID *id*, float * *quat*)

Retrieves the current orientation of a 3-Space Sensor relative to its tare orientation as a TSS_Quaternion. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
quat	The orientation will be written to this array as four floats. The order of elements will be x, y, z, w.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.113 EXPORT TSS_Error getFirstAvailableTSSComPort (ComPort * *tss_port*, int *filter*)

Scans the host computer for the first serial port it finds with a sensor (virtual or physical). Port is returned by tss_port. Calling getAvailableComPorts or getFirstAvailableTSSComPort will reset the iteration through the sensors.

Parameters

tss_port	A pointer to the first serial port with a 3-Space device that matches the filter.
filter	A mask of what 3-Space device types were desired.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.114 EXPORT TSS_Error getFirstAvailableUnknownComPort (ComPort * *ukn_port*)

Scans the host computer for the first serial port it finds with an unknown device (virtual or physical). Port is returned by ukn_port. Calling getAvailableComPorts or getFirstAvailableUnknownComPort will reset the iteration through the unknown ports.

Parameters

ukn_port	A pointer to the first serial port with an unknown device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.115 EXPORT TSS_Error getFriendlyName (TSS_ID *id*, char * *friendly_name*)

Retrieves the friendly name of the device's serial port as reported by the port's driver.

Parameters

id	The identifier for the 3-Space device.
friendly_name	The char buffer to write the string into. The string length can vary depending on the device. Make the buffer rather large (255 perhaps).

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.116 EXPORT TSS_Error getGyroCalibrationParam (TSS_ID *id,* TSS_Matrix * *mat,* TSS_Vector * *bias*)

Gets the gyro calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by is written to the referenced variable.
bias	The bias of the calibration is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.117 EXPORT TSS_Error getGyroCalibrationParamf9f3 (TSS_ID *id,* float * *mat,* float * *bias*)

Gets the gyro calibration parameters. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by is written to the referenced array of nine floats. The array is arranged in row-major order.
bias	The bias of the calibration is written to the referenced array of three floats. The array is arranged as: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.118 EXPORT TSS_Error getGyroEnabled (TSS_ID *id,* boolean * *is_enabled*)

Reads the enabled state of the gyros. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
is_enabled	The state of the gyro is written to the referenced variable. True if the gyro is enabled. False if the gyro is disabled.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.119 EXPORT TSS_Error getGyroNormalized (TSS_ID *id*, TSS_Vector * *gyro*)

Returns the normalized gyro rates of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
gyro	The normalized data from the 3-Space Sensor's on-board gyroscope is written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.120 EXPORT TSS_Error getGyroNormalizedf3 (TSS_ID *id*, float * *gyro*)

Returns the normalized gyro rates of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
gyro	The normalized data from the 3-Space Sensor's on-board gyroscope is written to the referenced array of 3 floats. The floats are ordered: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.121 EXPORT TSS_Error getGyroRange (TSS_ID *id*, unsigned char * *range*)

Reads the current operation range of the 3-Space Sensor's gyroscope. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
range	The range mode of the gyroscope is written to the referenced variable. Expected values are: 0 - +/- 250dps mode 1 - +/- 500dps mode 2 - +/- 2000dps mode

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.122 EXPORT TSS_Error getGyroRaw (TSS_ID *id*, TSS_Vector * *gyro*)

Returns the unaltered gyro rates of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
gyro	The unaltered data from the 3-Space Sensor's on-board gyroscope is written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.123 EXPORT TSS_Error getGyroRawf3 (TSS_ID *id*, float * *gyro*)

Returns the unaltered gyro rates of the 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
gyro	The unaltered data from the 3-Space Sensor's on-board gyroscope is written to the referenced array of 3 floats. The floats are ordered: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.124 EXPORT TSS_Error getHardwareVersion (TSS_ID *id*, char * *version*)

Reads the version and type of hardware being used on the 3-Space device.

Parameters

id	The identifier for the 3-Space device.
version	The 3-Space device's hardware version is written as a char array string of length 33 (counting the NULL terminator).

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.125 EXPORT TSS_Error getHIDUpdateRate (TSS_ID *id*, unsigned char * *rate_ms*)

Reads the update rate of the mouse and joystick HID devices. This call will error if the inputted id is not for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
rate_ms	The update rate of HID device polling in milliseconds is written to the referenced variable. The minimum valid value is 5 milliseconds, while the maximum valid value is 50 milliseconds. The dongle's default rate is 25.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.126 EXPORT TSS_Error getInformation (TSS_ID *id*, BootloaderData * *data*)

Gets information about the 3-Space device in bootloader mode. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

Parameters

id	The identifier for the 3-Space device.
data	The information about the device will be written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.127 EXPORT TSS_Error getInterruptStatus (TSS_ID *id*, unsigned char * *status*)

Read the current interrupt state. Calling this command while interrupts are in mode '2' (see definition for setInterruptType for more details) will cause the interrupt pin to return to high. This call will error if the inputted id is not for a 3-Space Embedded Sensor.

Parameters

id	The identifier for the 3-Space device.
status	The value that specifies the sensor's interrupt status will be written to the referenced variable. An integer whose value can either be 0 or 1. If the value is 0, there is no new data since the last orientation read. If the value is 1, then new data is available.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.128 EXPORT TSS_Error getInterruptType (TSS_ID *id*, unsigned char * *mode*, unsigned char * *pin*)

Returns how the sensor currently generates interrupts. This call will error if the inputted id is not for a 3-Space Embedded Sensor.

Parameters

id	The identifier for the 3-Space device.
mode	The value that specifies how the sensor will generate the interupt and for how long will be written to the referenced variable. If mode is 0, no generation will occur. If mode is 1, the interrupt generator will set the specified pin low for 5 microseconds when new data is available. If mode is 2, the interrupt generator will set the pin low until the interrupt status is read with the getInterruptStatus function.
pin	The value that defines what pin on the sensor to generate the interupt on will be written to the referenced variable. If pin is 0, the TXD pin is used. If pin is 1, the MISO pin is used to generate the interrupt. The method of communication each pin belongs to cannot be uwed while that pin is being used for interrupt generation. Note that the pin cannot be changed to TXD over the UART, and cannot be changed to MISO over SPI.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.129 EXPORT TSS_Error getJoystickEnabled (TSS_ID *id,* boolean * *is_enabled*)

Reads the enabled status for the HID Joystick for a 3-Space device. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
is_enabled	The joystick state is written to the referenced variable. If True, the joystick is enabled. If False, the joystick is disabled.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.130 EXPORT TSS_Error getJoystickLogicalID (TSS_ID *id,* unsigned char * *logical_id*)

Reads the logical ID of the wireless unit will operate as the joystick. This call will error if the inputted id is not for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
logical_id	The logical ID (index in the dongle's wireless hardware table) associated with the wireless sensor that is to act as the data source for joystick HID streaming is written to the referenced variable. If this value is set to -1, there will be no joystick input from any wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.131 EXPORT TSS_Error getJoystickMousePresent (TSS_ID *id,* boolean * *is_joy_present,* boolean * *is_mouse_present*)

Reads whether the joystick and mouse are present or removed.

If removed, they will not show up as devices on the target system at all.

Parameters

id	The identifier for the 3-Space device.
is_joy_present	The state indicating whether the joystick is present on the system at all or not is written to the referenced variable. If True, the joystick is present. If False, the joystick is not present.
is_mouse_present	The state indicating whether the mouse is present on the system at all or not is written to the referenced variable. If True, the mouse is present. If False, the mouse is not present.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.132 EXPORT TSS_Error getKalmanMat (TSS_ID *id,* TSS_Matrix * *mat*)

Read Kalman filter's covariance 3x3 matrix. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mat	The covariance matrix is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.133 EXPORT TSS_Error getKalmanMatf9 (TSS_ID *id,* float * *mat*)

Read Kalman filter's covariance 3x3 matrix. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mat	The covariance matrix is written to the referenced array of nine floats. The floats in the array are ordered in row-major order.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.134 EXPORT TSS_Error getLEDColor (TSS_ID *id,* TSS_Color * *color*)

Reads the color of the LED on the sensor.

Parameters

id	The identifier for the 3-Space device.
color	The color of the device's LED is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.135 EXPORT TSS_Error getLEDColorf3 (TSS_ID *id,* float * *color*)

Reads the color of the LED on the sensor.

Parameters

id	The identifier for the 3-Space device.
color	The color of the device's LED is written to an array of three floats. Each float in the array has an expected range of 0.0 (no color contribution) to 1.0 (greatest color contribution). The first float is the red channel, the second the green channel, and the third float is the blue channel. (red, green, blue)

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.136 EXPORT TSS_Error getLEDMode (TSS_ID *id,* unsigned char * *mode*)

Reads the mode of the device's LED. This call will error if the inputted id is not for a 3-Space Dongle, 3-Space Wireless Sensor, or a 3-Space Bluetooth Sensor.

The LED has two possible mode, 'static' and standard.

- If the LED is in 'static' mode, this means that it will only display the color set by the command setLEDColor.
- If the LED is in 'standard' mode, it will display the standard LED colors as described below:
  - Upon receipt of a packet, the dongle will flash green temporarily.
  - If the dongle transmits a packet that does not reach its destination, the dongle will flash red temporarily.

Parameters

id	The identifier for the 3-Space device.
mode	The device's LED mode will be written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.137 EXPORT TSS_Error getLookupTblVertVal (TSS_ID *id,* unsigned char *type,* unsigned short *idx,* TSS_Vector * *val*)

Reads a vector entry in a 3-Space Sensor's vertex look-up table. This call will error if the inputted id is for a 3-Space Dongle.

This function is an experimental calibration function that is not guarenteed to be supported.

Parameters

id	The identifier for the 3-Space device.
type	The type of on-board sensor data to log. Possible values are: 0 - Compass 1 - Accelerometer
idx	The index into the lookup table to write to. The value may range from 0 to 1352.
val	The vector stored in the table will be written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.138 EXPORT TSS_Error getLookupTblVertValf3 (TSS_ID *id,* unsigned char *type,* unsigned short *idx,* float * *val*)

Reads a vector entry in a 3-Space Sensor's vertex look-up table. This call will error if the inputted id is for a 3-Space Dongle.

This function is an experimental calibration function that is not guarenteed to be supported.

Parameters

id	The identifier for the 3-Space device.
type	The type of on-board sensor data to log. Possible values are: 0 - Compass 1 - Accelerometer
idx	The index into the lookup table to write to. The value may range from 0 to 1352.
val	The vector stored in the table will be written to the referenced array of three floats. The array will be arranged: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.139 EXPORT TSS_Error getMouseEnabled (TSS_ID *id,* boolean * *is_enabled*)

Turns the data feed to the mouse on and off. This call will error if the inputted id is for a 3-Space Dongle.

If disabled, the sensor will still enumerate as a mouse, but the mouse will not function. This allows normal communication to occur at a faster rate.

Parameters

id	The identifier for the 3-Space device.
is_enabled	The mouse state is written to the referenced variable. If True, the mouse is enabled. If False, the mouse is disabled.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.140 EXPORT TSS_Error getMouseLogicalID (TSS_ID *id,* unsigned char * *logical_id*)

Reads the logical ID of the wireless unit will operate as the mouse. This call will error if the inputted id is not for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
logical_id	The logical ID (index in the dongle's wireless hardware table) associated with the wireless sensor that is to act as the data source for mouse HID streaming is written to the referenced variable. If this value is set to -1, there will be no mouse input from any wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.141 EXPORT TSS_Error getMouseRelative (TSS_ID *id,* boolean * *is_relative*)

Reads the current emulated HID mouse absolute/relative state.

Note that if the sensor has not been reset since it has been put in this state, the mouse will not reflect this change yet, even though this command will.

Parameters

id	The identifier for the 3-Space device.
is_relative	The absolute/relative state of the mouse is written to the referenced variable. If the value is True, the mouse is relative. If the value is False, the mouse is in absolute mode.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.142 EXPORT TSS_Error getMultiRefChkVecAccelerometer (TSS_ID *id,* unsigned char *idx,* TSS_Vector * *vec*)

Reads the multi mode accelerometer reference check vector at index. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the desired reference check vector.
vec	The reference check vector is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.143 EXPORT TSS_Error getMultiRefChkVecAccelerometerf3 (TSS_ID *id*, unsigned char *idx*, float * *vec*)

Reads the multi mode accelerometer reference check vector at index. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the desired reference check vector.
vec	The reference check vector is written to the referenced array of three floats. The array is structured: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.144 EXPORT TSS_Error getMultiRefChkVecCompass (TSS_ID *id*, unsigned char *idx*, TSS_Vector * *vec*)

Reads the multi mode compass reference check vector at index. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the desired reference check vector.
vec	The reference check vector is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.145 EXPORT TSS_Error getMultiRefChkVecCompassf3 (TSS_ID *id*, unsigned char *idx*, float * *vec*)

Reads the multi mode compass reference check vector at index. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the desired reference check vector.
vec	The reference check vector is written to the referenced array of three floats. The array is structured: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.146 EXPORT TSS_Error getMultiRefResolution (TSS_ID *id,* unsigned char * *num_division,* unsigned char * *num_vecs*)

Reads number of cell divisions and number of nearby vectors per cell for the multi reference lookup table. This call will error if the inputted id is for a 3-Space Dongle.

By default, the multiple reference vector mode only deals with orientations obtainable by successive rotations of 90 degrees about any of the three axes. This command can adjust it to accept orientations obtainable by 45 degree rotations. For 90 degrees, give a "number of cell divisions" of 4, and for 45 give 8. In addition, the number of vectors near to any given orientation which the scheme will check can be adjusted as well. If this value is 0, only the nearest orientation will be checked. The largest this value can be is 32. Also note that the larger this value is, the longer the multiple reference vector mode will take to run each cycle.

Parameters

id	The identifier for the 3-Space device.
num_division	The number of cell divisions is written to the referenced variable.
num_vecs	The number of nearby vectors is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.147 EXPORT TSS_Error getMultiRefVecAccelerometer (TSS_ID *id,* unsigned char *idx,* TSS_Vector * *vec*)

Reads the multi mode accelerometer reference vector at index. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the desired reference vector.
vec	The reference vector is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.148 EXPORT TSS_Error getMultiRefVecAccelerometerf3 (TSS_ID *id,* unsigned char *idx,* float * *vec*)

Reads the multi mode accelerometer reference vector at index. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the desired reference vector.
vec	The reference vector is written to the referenced array of three floats. The array is structured: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.149 EXPORT TSS_Error getMultiRefVecCompass (TSS_ID *id*, unsigned char *idx*, TSS_Vector * *vec*)

Reads the multi mode compass reference vector at index. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the desired reference vector.
vec	The reference vector is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.150 EXPORT TSS_Error getMultiRefVecCompassf3 (TSS_ID *id*, unsigned char *idx*, float * *vec*)

Reads the multi mode compass reference vector at index. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the desired reference vector.
vec	The reference vector is written to the referenced array of three floats. The array is structured: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.151 EXPORT TSS_Error getMultiRefWeightPwr (TSS_ID *id*, float * *pow*)

Get the weighting power for multi reference vector weights. This call will error if the inputted id is for a 3-Space Dongle.

Multi reference vector weights are all raised to the weight power before they are summed and used in the calculation for the final reference vector. Setting this value nearer to 0 will cause the reference vectors to overlap more, and setting it nearer to infinity will cause the reference vectors to influence a smaller set of orientations.

Parameters

id	The identifier for the 3-Space device.
pow	The weight power to use.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.152 EXPORT TSS_Error getNextAvailableTSSComPort (ComPort * *tss_port*)

Call after getFirstAvailableTSSComPort to get more sensors. Port is returned by tss_port. getFirstAvailableTSSComPort must be called before calling getNextAvailableTSSComPort Calling getAvailableComPorts or getFirstAvailableTSSComPort will reset the iteration through the sensors.

Parameters

tss_port	A pointer to the next serial port with a 3-Space device that matches the filter used in getFirstAvailableTSSComPort.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.153 EXPORT TSS_Error getNextAvailableUnknownComPort (ComPort * *ukn_port*)

Call after getFirstAvailableUnknownComPort to get more sensors. Port is returned by tss_port. getFirstAvailableUnknownComPort must be called before calling getNextAvailableUnknownComPort Calling getAvailableComPorts or getFirstAvailableUnknownComPort will reset the iteration through the unknown ports.

Parameters

ukn_port	A pointer to the next serial port with an unknown device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.154 EXPORT TSS_Error getNumMultiRefCells (TSS_ID *id,* unsigned int * *num_cells*)

Read number of multi reference cells. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
num_cells	The number of multi reference cells is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.155 EXPORT TSS_Error getOversampleRate (TSS_ID *id,* unsigned char * *rate*)

Read oversample rate. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
rate	The oversample rate is written to the referenced structure.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.156 EXPORT TSS_Error getPanID (TSS_ID *id*, unsigned short * *pan_id*)

Reads the Pan ID for the 3-Space Device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.

Parameters

id	The identifier for the 3-Space device.
pan_id	The Pan ID of the 3-Space device is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.157 EXPORT TSS_Error getReceptionStrength (TSS_ID *id*, float * *recep_strg*)

Reads the relative strength of the most recent packet received by the dongle. This call will error if the inputted id is not for a 3-Space Dongle.

Reads a value indicating the relative strength of the most recent packet received by the dongle. The value scales linearly with distance from the dongle. Low values do not necessarily mean that transmissions are less accurate, but rather the strength of the signal that transmitted the last packet was weak.

Parameters

id	The identifier for the 3-Space device.
recep_strg	The signal strength of the last wireless packet recieved from a wireless sensor is written to the referenced variable. The value ranges from 0 to 255.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.158 EXPORT TSS_Error getRefVecAccelerometer (TSS_ID *id*, TSS_Vector * *vec*)

Reads the single mode accelerometer reference vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
vec	The reference vector is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.159 EXPORT TSS_Error getRefVecAccelerometerf3 (TSS_ID *id*, float * *vec*)

Reads the single mode accelerometer reference vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
vec	The reference vector is written to the referenced array of three floats. The array is formatted as such: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.160 EXPORT TSS_Error getRefVecCompass (TSS_ID *id*, TSS_Vector * *vec*)

Reads the single mode compass reference vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
vec	The reference vector is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.161 EXPORT TSS_Error getRefVecCompassf3 (TSS_ID *id*, float * *vec*)

Reads the single mode compass reference vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
vec	The reference vector is written to the referenced array of three floats. The array is formatted as such: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.162 EXPORT TSS_Error getRefVecMode (TSS_ID *id*, unsigned char * *mode*)

Reads the reference vector mode. This call will error if the inputted id is for a 3-Space Dongle.

Single static mode uses a certain reference vector for the compass and another certain reference vector for the accelerometer at all times.

Single auto mode uses (0,-1,0) as the reference vector for the accelerometer at all times and uses the current average angle between the accelerometer and compass to calculate the compass reference vector. After that this mode acts like single static mode.

Single auto continuous mode uses (0,-1,0) as the reference vector for the accelerometer at all times and uses the average angle between the accelerometer and compass to constantly redetermine the compass reference vector.

Multi mode uses a collection of reference vectors for the compass and accelerometer, and selects which ones to use before each step of the filter.

Parameters

id	The identifier for the 3-Space device.
mode	The reference vector mode is written to the referenced variable. mode will be one of four possible values: 0 - single static. 1 - single auto. 2 - single auto continuous. 3 - multi.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.163 EXPORT TSS_Error getRhoDataAccelerometer (TSS_ID *id,* unsigned char * *mode,* float * *min_static,* float * *max*)

Reads the rho mode and rho data for the accelerometer. Rho mode, min/static rho, and the max rho (if the rho mode is 'confidence') are returned. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mode	The rho mode mode. If the value is 0, the rho mode is 'static'. If the value is 1, the rho mode is 'confidence'.
min_static	If mode is static, this is the rho value. If mode is confidence, this is the minimum rho value.
max	If mode is static, zero will be written to this variable. If mode is confidence, this is the maximum rho value.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.164 EXPORT TSS_Error getRhoDataCompass (TSS_ID *id,* unsigned char * *mode,* float * *min_static,* float * *max*)

Reads the rho mode and rho data for the compass. Rho mode, min/static rho, and the max rho (if the rho mode is 'confidence') are returned. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mode	The rho mode mode. If the value is 0, the rho mode is 'static'. If the value is 1, the rho mode is 'confidence'.
min_static	If mode is static, this is the rho value. If mode is confidence, this is the minimum rho value.
max	If mode is static, zero will be written to this variable. If mode is confidence, this is the maximum rho value.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.165 EXPORT TSS_Error getRunningAvgMode (TSS_ID *id*, unsigned char * *mode*)

Reads the mode that the running-average method uses. This call will error if the inputted id is for a 3-Space Dongle.

'Normal' uses a static running-average. 'Confidence' uses a running average that changes dynamically based upon the confidence factor.

Parameters

id	The identifier for the 3-Space device.
mode	The sensor's running average mode is written to the referenced variable. Possible values are: 0 - Normal mode 1 - Confidence Mode

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.166 EXPORT TSS_Error getSensorFromDongle (TSS_ID *id*, unsigned int *idx*, TSS_ID * *wl_id*)

Retrieves the unique identifier for a 3-Space Wireless Sensor already registered with a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space Dongle.
idx	The index into the dongle's wireless table to register the wireless sensor to. May be between 0 and 15.
wl_id	The unique identifier for the wireless sensor will be written to the variable referenced by this pointer.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.167 EXPORT TSS_Error getSerialNumber (TSS_ID *id*, unsigned int * *serial*)

Reads the serial number of the sensor.

The serial number is a unique number for any one sensor.

Parameters

id	The identifier for the 3-Space device.
serial	The serial number of the 3-Space device is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.168 EXPORT TSS_Error getSerialNumberHex (TSS_ID *id*, char * *hex_serial*)

Retrieves a hexidecimal string representation of the 3-Space device's serial number (matching the representation on the case of the device).

Parameters

id	The identifier for the 3-Space device.
hex_serial	The char buffer to write the string into. Should have a length of 9 (8 for the hex string, 1 for a null terminator).

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.169 EXPORT TSS_Error getSoftwareVersion (TSS_ID *id*, char * *version*)

Reads the version of the currenly installed firmware on the 3-Space device.

Parameters

id	The identifier for the 3-Space device.
version	The 3-Space device's firmware version is written as a char array string of length 13 (counting the NULL terminator).

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.170 EXPORT TSS_Error getTareOrientMat (TSS_ID *id*, TSS_Matrix * *mat*)

Reads the tare orientation as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mat	The structure the tare orientation is written to.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.171 EXPORT TSS_Error getTareOrientMatf9 (TSS_ID *id*, float * *mat*)

Reads the tare orientation as a rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mat	The array of nine floats the tare orientation is written to. The array is in row major.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.172 EXPORT TSS_Error getTareOrientQuat (TSS_ID *id*, TSS_Quaternion * *quat*)

Reads the tare orientation as a quaternion. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
quat	The structure the tare oriention is written to.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.173 EXPORT TSS_Error getTareOrientQuatf4 (TSS_ID *id*, float * *quat*)

Reads the tare orientation as a quaternion. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
quat	The array of four floats the tare oriention is written to. The array is structured: x, y, z, w.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.174 EXPORT TSS_Error getTemperatureCelsius (TSS_ID *id*, float * *temp_c*)

Returns the temperature of the sensor in Celsius. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
temp_c	The temprature read by the sensor's thermometer in Celsius is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.175 EXPORT TSS_Error getTemperatureFahrenheit (TSS_ID *id*, float * *temp_f*)

Returns the temperature of the sensor in Fahrenheit. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
temp_f	The temprature read by the sensor's thermometer in Fahrenheit is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.176 EXPORT TSS_Error getUARTRate (TSS_ID *id,* unsigned int * *rate*)

Reads the baud rate of the physical UART. This call will error if the inputted id is not for a 3-Space USB Sensor or a 3-Space Embedded Sensor.

If the UART rate has been changed, but the sensor has not been reset, the read rate will reflect the newly set rate as opposed to the current operating rate.

Parameters

id	The identifier for the 3-Space device.
rate	The current set baud rate of the UART will be written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.177 EXPORT TSS_Error getUSBMode (TSS_ID *id,* unsigned char * *mode*)

Gets the communication mode for USB.

All modes present a COM port with which to communicate with the USB device. FTDI and CDC each present a regular numbered port, whereas Unique presents a port named YEI_<serial number>. The sensor will change modes immediately.

Parameters

id	The identifier for the 3-Space device.
mode	The USB mode for the sensor is written to the referenced variable. Valid values are: • 0 for CDC mode. • 1 for FTDI mode. • 2 for Unique mode.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.178 EXPORT TSS_Error getWirelessAddress (TSS_ID *id,* unsigned short * *address*)

Reads the wireless address of the device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.

Parameters

id	The identifier for the 3-Space device.
address	The device's wireless address will be written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.179 EXPORT TSS_Error getWirelessHWID (TSS_ID *id,* unsigned char *idx,* unsigned int * *hw_id*)

Reads the hardware ID (sensor serial number) that is associated with the logical ID given by an index. This call will error if the inputted id is not for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the dongle's wireless table to read the hardware ID from.
hw_id	The serial number of the wireless sensor currently listed in the dongle's wireless table under idx is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.180 EXPORT TSS_Error getWirelessRetries (TSS_ID *id,* unsigned char * *retries*)

Reads the number of times the dongle will retry a failed transmission to a wireless sensor. This call will error if the inputted id is not for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
retries	The number of times to retry a transmission to a wireless sensor before giving up is written to the referenced variable.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.181 EXPORT TSS_Error getWirelessSlotsOpen (TSS_ID *id,* unsigned char * *slots*)

Reads how many more commands slots are available in the dongle's wireless send queue. This call will error if the inputted id is not for a 3-Space Dongle.

All commands sent to the dongle to be transmitted wirelessly are buffered until previous pending transmissions complete. There are fifteen such slots, and while the dongle handles the management of them internally, no additional transmissions can be queued up if the maximum number of slots are occupied already. If sending a large batch of commands to a single sensor, it is useful to check this to make sure the transmission can be initiated.

Parameters

id	The identifier for the 3-Space device.
slots	The number of available wireless slots is written to the referenced variable. The maximum number of slots possible is 15.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.182 EXPORT boolean isConnected (TSS_ID *id,* boolean *try_reconnect*)

Checks if a 3-Space device is currently connected to the computer.

Parameters

id	The identifier for the device to check.
try_reconnect	If True, an attempt will be made to reconnect the device if the device is not connected.

Returns

True if the device is (re)connected. False if the device is not connected.

4.1.4.183 EXPORT TSS_Error isFirmwareValid (TSS_ID *id,* boolean * *is_valid*)

Checks the firmware loaded into the 3-Space device is valid or not. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

Parameters

id	The identifier for the 3-Space device.
is_valid	If True is written to this referenced variable, the firmware is valid. If False is written, the firmware is corrupt in some way.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.184 EXPORT TSS_Error resetKalmanFilter (TSS_ID *id*)

Resets Kalman filter's covariance and state matrices. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.185 EXPORT TSS_Error restoreFactorySettings (TSS_ID *id*)

Restores all settings to factory settings.

The settings are not committed to non-volatile memory by this command, so the commit settings command will have to be used if this is desired.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.186 EXPORT TSS_Error setAccelerometerCalibrationParam (TSS_ID *id,* TSS_Matrix *mat,* TSS_Vector *bias*)

Sets the accelerometer calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by.
bias	The vector indicating the bias of the calibration.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.187 EXPORT TSS_Error setAccelerometerCalibrationParamf9f3 (TSS_ID *id,* float * *mat,* float * *bias*)

Sets the accelerometer calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by as an array of nine floats. The floats should be arranged in row-major order.
bias	The vector indicating the bias of the calibration as an array of three floats. The floats should be arranged as: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.188 EXPORT TSS_Error setAccelerometerEnabled (TSS_ID *id,* boolean *do_enable*)

Enables or disables the accelerometer (the filter will not use this data if disabled). This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
do_enable	Whether to enable or disable the accelerometer (False - Disabled, True - Enabled).

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.189 EXPORT TSS_Error setAccelerometerRange (TSS_ID *id,* unsigned char *range*)

Set which range of accelerometer data to use. Higher ranges can detect and report larger accelerations, but are not as accurate for smaller ones. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
range	The intended range for the accelerometer. Valid values are: 0 for 2G, 1 for 4G, and 2 for 8G.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.190 EXPORT TSS_Error setAvgPercent (TSS_ID *id,* float *val*)

Sets what percentage of running average to use on the sensor's orientation.

This is computed as follows:

total_orient = total_orient $*$ percent

total_orient = total_orient + current_orient $*$ (1 - percent)

current_orient = total_orient

If the percentage is 0, the running average will be shut off completely.

This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
val	The percentage of the running average to use for the sensor's orientation. Valid values are within the range 0.0 to 1.0.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.191 EXPORT TSS_Error setAxisDirections (TSS_ID *id,* AxisDirections *axis_dir*)

Sets the axis direction configuration for a 3-Space Sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
axis_dir	The axis direction configuration to set the sensor to.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.192 EXPORT TSS_Error setButtonGyroDisableLength (TSS_ID *id,* unsigned char *length*)

Sets how long, in frames, the gyros should be disabled after one of the physical buttons on the sensor is pressed. This call will error if the inputted id is for a 3-Space Dongle.

This setting helps to alleviate gyro disturbances caused by the buttons causing small shockwaves in the sensor.

Parameters

id	The identifier for the 3-Space device.
length	How many frames to disable the gyros for when a button is pressed. A setting of 0 means the gyros won't be disabled at all.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.193 EXPORT TSS_Error setChannel (TSS_ID *id,* unsigned char *channel*)

Sets the wireless channel for the 3-Space Device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.

Parameters

id	The identifier for the 3-Space device.
channel	The wireless channel to set to the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.194 EXPORT TSS_Error setClock (TSS_ID *id,* TSS_Clock *clock_data*)

Sets the current data and time of the 3-Space Sensor's internal clock to the inputted date and time. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.

Parameters

id	The identifier for the 3-Space device.
clock_data	The current time to set the clock to.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.195 EXPORT TSS_Error setClockc6 (TSS_ID *id,* unsigned char * *clock_data*)

Sets the current data and time of the 3-Space Sensor's internal clock to the inputted date and time. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.

Parameters

id	The identifier for the 3-Space device.
clock_data	The current time to set the clock to as an array of six chars. This array should be formatted as: month, day, year, hour, minute, second. Reference TSS_Clock for a definition of the expected ranges of values for the array elements.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.196 EXPORT TSS_Error setClockSpeed (TSS_ID *id,* unsigned int *speed*)

Sets the clock speed of the device's MCU to the given value.

Parameters

id	The identifier for the 3-Space device.
speed	The desired clock speed of the MCU in Hz. As an example, if one desired to set the clock speed to 15 MHz, the speed argument should be set to 15000000. Valid values are: • 15000000 (15 MHz). • 30000000 (30 MHz). • 60000000 (60 MHz).

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.197 EXPORT TSS_Error setCompassCalibrationParam (TSS_ID *id,* TSS_Matrix *mat,* TSS_Vector *bias*)

Sets the compass calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by.
bias	The vector indicating the bias of the calibration.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.198 EXPORT TSS_Error setCompassCalibrationParamf9f3 (TSS_ID *id,* float * *mat,* float * *bias*)

Sets the compass calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by as an array of nine floats. The floats should be arranged in row-major order.
bias	The vector indicating the bias of the calibration as an array of three floats. The floats should be arranged as: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.199 EXPORT TSS_Error setCompassEnabled (TSS_ID *id,* boolean *do_enable*)

Enables or disables the compass (the filter will not use this data if disabled). This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
do_enable	Whether to enable or disable the compass (False - Disabled, True - Enabled).

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.200 EXPORT TSS_Error setCompassRange (TSS_ID *id,* unsigned char *range*)

Sets the measurement range used by the compass sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
range	The range mode to set the compass to. Possible values are: 0 - +/- 0.88Ga mode 1 - +/- 1.30Ga mode 2 - +/- 1.90Ga mode 3 - +/- 2.50Ga mode 4 - +/- 4.00Ga mode 5 - +/- 4.70Ga mode 6 - +/- 5.60Ga mode 7 - +/- 8.10Ga mode

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.201 EXPORT TSS_Error setConfidenceRhoModeAccel (TSS_ID *id,* float *min,* float *max*)

Sets the rho mode for the accelerometer to confidence, using the given values as the min and the max. The confidence factor will be used in real time to interpolate between the min and max to determine what rho value is used. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
min	Represents the minimum Rho value to be used for calculation.
max	Represents the maximmum Rho value to be used for calculation.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.202 EXPORT TSS_Error setConfidenceRhoModeCompass (TSS_ID *id,* float *min,* float *max*)

Sets the rho mode for the compass to confidence, using the given values as the min and the max. The confidence factor will be used in real time to interpolate between the min and max to determine what rho value is used. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
min	Represents the minimum Rho value to be used for calculation.
max	Represents the maximmum Rho value to be used for calculation.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.203 EXPORT TSS_Error setFilterMode (TSS_ID *id,* unsigned char *mode*)

Used to disable the orientation filter or set the orientation filter mode. Changing this parameter can be useful for tuning filter-performance versus orientation-update rates. When using the sensor as an IMU, it will improve performance to disable the orientation filter. When using as an AHRS, where orientation is needed, full-Kalman, gyroless-filtered, or fast-filtered should be used. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mode	The mode to set the sensor's filter to. Possible values are: 0 - Disables filter 1 - Full Kalman Filter 2 - Gyroless Filtered Orientation *Not Yet Supported* 3 - Fast Filtered Orientation *Not Yet Supported*

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.204 EXPORT TSS_Error setGlobalAxis (TSS_ID *id,* char * *joy_or_mouse,* char * *device_axis,* TSS_Vector *local_axis,* TSS_Vector *global_axis,* float *deadzone,* float *scale,* float *power*)

Sets an axis of the desired emulated input device as a 'Global Axis' style axis. This call will error if the inputted id is for a 3-Space Dongle.

Axis operating under this style use a reference vector and a consitent local vector to determine the state of the device's axis. As the local vector rotates, it is projected onto the global vector. Once the distance of that projection on the global vector exceeds the inputted "deadzone", the device will begin tranmitting non-zero values for the device's desired axis.

Parameters

id	The identifier for the 3-Space device.
joy_or_mouse	A null-terminated string whose value may be either 'mouse' or 'joystick'. This string defines whether the device in question is a mouse or joystick.
device_axis	A null-terminated string whose value may be either 'X' or 'Y'. This string defines what axis of the device is to be configured.
local_axis	A normalized vector representing the sensor's local vector to track.
global_axis	A normalized vector representing the global vector to project the local vector onto (should be orthoginal to the local vector).
deadzone	Defines the minimum distance necissary for the device's axis to read a non-zero value.
scale	Defines the linear scale for the values being returned for the axis.
power	An exponental power used to further modify data being returned from the sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.205 EXPORT TSS_Error setGyroCalibrationParam (TSS_ID *id,* TSS_Matrix *mat,* TSS_Vector *bias*)

Sets the gyro calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by.
bias	The vector indicating the bias of the calibration.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.206 EXPORT TSS_Error setGyroCalibrationParamf9f3 (TSS_ID *id,* float * *mat,* float * *bias*)

Sets the gyro calibration parameters to the given values. This call will error if the inputted id is for a 3-Space Dongle.

These consist of a bias which is applied to the raw data as a translation, and a matrix by which the value is multiplied by.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to multiply the raw data by as an array of nine floats. The floats should be arranged in row-major order.
bias	The vector indicating the bias of the calibration as an array of three floats. The floats should be arranged as: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.207 EXPORT TSS_Error setGyroEnabled (TSS_ID *id,* boolean *do_enable*)

Enables or disables the gyros (will be replaced with a still gyro reading if disabled). This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
do_enable	Whether to enable or disable the gyro (False - Disabled, True - Enabled).

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.208 EXPORT TSS_Error setGyroRange (TSS_ID *id,* unsigned char *range*)

Sets the measurment range used by the gyroscope sensor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
range	The range mode to set the gyroscope to. Possible values are: 0 - +/- 250dps mode 1 - +/- 500dps mode 2 - +/- 2000dps mode

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.209 EXPORT TSS_Error setHIDUpdateRate (TSS_ID *id,* unsigned char *rate_ms*)

Sets the update rate of the mouse and joystick HID devices. This call will error if the inputted id is not for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
rate_ms	The intended update rate of HID device polling in milliseconds. The minimum valid value is 5 milliseconds, while the maximum valid value is 50 milliseconds. The dongle's default rate is 25.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.210 EXPORT TSS_Error setInterruptType (TSS_ID *id,* unsigned char *mode,* unsigned char *pin*)

Configures how the sensor will generate interrupts.. This call will error if the inputted id is not for a 3-Space Embedded Sensor.

Parameters

id	The identifier for the 3-Space device.
mode	A value that specifies how the sensor will generate the interupt and for how long. If mode is 0, no generation will occur. If mode is 1, the interrupt generator will set the specified pin low for 5 microseconds when new data is available. If mode is 2, the interrupt generator will set the pin low until the interrupt status is read with the getInterruptStatus function.
pin	A value that defines what pin on the sensor to generate the interupt on. If pin is 0, the TXD pin is used. If pin is 1, the MISO pin is used to generate the interrupt. The method of communication each pin belongs to cannot be uwed while that pin is being used for interrupt generation. Note that the pin cannot be changed to TXD over the UART, and cannot be changed to MISO over SPI.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.211 EXPORT TSS_Error setJoystickEnabled (TSS_ID *id,* boolean *do_enabled*)

Turns the data feed to the joystick on and off. This call will error if the inputted id is for a 3-Space Dongle.

If disabled, the sensor will still enumerate as a joystick, but the joystick will not function. This allows normal communication to occur at a faster rate.

Parameters

id	The identifier for the 3-Space device.
do_enabled	If True, the joystick is enabled. If False, the joystick is disabled.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.212 EXPORT TSS_Error setJoystickLogicalID (TSS_ID *id,* unsigned char *logical_id*)

Set the logical ID of the wireless unit will operate as the joystick or disables joystick data transmission. This call will error if the inputted id is not for a 3-Space Dongle.

Disabling joystick data transmission allows normal communication to occur at a faster rate.

Parameters

id	The identifier for the 3-Space device.
logical_id	The logical ID (index in the dongle's wireless hardware table) associated with the wireless sensor that is to act as the data source for joystick HID streaming. If this value is set to -1, there will be no joystick input from any wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.213 EXPORT TSS_Error setJoystickMousePresent (TSS_ID *id,* boolean *do_joy_present,* boolean *do_mouse_present*)

Sets whether the joystick and mouse are present or removed.

If removed, they will not show up as devices on the target system at all. For these changes to take effect, the sensor driver may need to be reinstalled.

Parameters

id	The identifier for the 3-Space device.
do_joy_present	States whether the joystick is present on the system at all. If True, the joystick is present. If False, the joystick is not present.
do_mouse_present	States whether the mouse is present on the system at all. If True, the mouse is present. If False, the mouse is not present.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.214 EXPORT TSS_Error setLEDColor (TSS_ID *id,* TSS_Color *color*)

Sets the color of the LED on the sensor to the given RGB color.

Parameters

id	The identifier for the 3-Space device.
color	The color the device's LED is being set to.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.215 EXPORT TSS_Error setLEDColorf3 (TSS_ID *id,* float * *color*)

Sets the color of the LED on the sensor to the given RGB color.

Parameters

id	The identifier for the 3-Space device.
color	The color the device's LED is being set to as an array of three floats. Each float in the array has an expected range of 0.0 (no color contribution) to 1.0 (greatest color contribution). The first float is the red channel, the second the green channel, and the third float is the blue channel. (red, green, blue)

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.216 EXPORT TSS_Error setLEDMode (TSS_ID *id,* unsigned char *mode*)

Sets the mode of the device's LED. This call will error if the inputted id is not for a 3-Space Dongle, 3-Space Wireless Sensor, or a 3-Space Bluetooth Sensor.

The LED has two possible mode, 'static' and standard.

- If the LED is in 'static' mode, this means that it will only display the color set by the command setLEDColor.
- If the LED is in 'standard' mode, it will display the standard LED colors as described below:
  - Upon receipt of a packet, the dongle will flash green temporarily.
  - If the dongle transmits a packet that does not reach its destination, the dongle will flash red temporarily.

Parameters

id	The identifier for the 3-Space device.
mode	The LED mode to set the device to. If 0, the dongle will be set to 'standard' LED mode. If 1, the dongle will be set to 'static' LED mode.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.217 EXPORT TSS_Error setLookupTblVertVal (TSS_ID *id,* unsigned char *type,* unsigned short *idx,* TSS_Vector *val*)

Set a vector entry in a 3-Space Sensor's vertex look-up table. This call will error if the inputted id is for a 3-Space Dongle.

This function is an experimental calibration function that is not guarenteed to be supported.

Parameters

id	The identifier for the 3-Space device.
type	The type of on-board sensor data to log. Possible values are: 0 - Compass 1 - Accelerometer
idx	The index into the lookup table to write to. The value may range from 0 to 1352.
val	The vector to store in the table.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.218 EXPORT TSS_Error setLookupTblVertValf3 (TSS_ID *id,* unsigned char *type,* unsigned short *idx,* float * *val*)

Set a vector entry in a 3-Space Sensor's vertex look-up table. This call will error if the inputted id is for a 3-Space Dongle.

This function is an experimental calibration function that is not guarenteed to be supported.

Parameters

id	The identifier for the 3-Space device.
type	The type of on-board sensor data to log. Possible values are: 0 - Compass 1 - Accelerometer
idx	The index into the lookup table to write to. The value may range from 0 to 1352.
val	The vector to store in the table as an array of three floats. The array should be arranged: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.219 EXPORT TSS_Error setMouseEnabled (TSS_ID *id,* boolean *do_enabled*)

Turns the data feed to the mouse on and off. This call will error if the inputted id is for a 3-Space Dongle.

If disabled, the sensor will still enumerate as a mouse, but the mouse will not function. This allows normal communication to occur at a faster rate.

Parameters

id	The identifier for the 3-Space device.
do_enabled	If True, the mouse is enabled. If False, the mouse is disabled.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.220 EXPORT TSS_Error setMouseLogicalID (TSS_ID *id,* unsigned char *logical_id*)

Set the logical ID of the wireless unit will operate as the mouse or disables mouse data transmission. This call will error if the inputted id is not for a 3-Space Dongle.

Disabling mouse data transmission allows normal communication to occur at a faster rate.

Parameters

id	The identifier for the 3-Space device.
logical_id	The logical ID (index in the dongle's wireless hardware table) associated with the wireless sensor that is to act as the data source for mouse HID streaming. If this value is set to -1, there will be no mouse input from any wireless sensor.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.221 EXPORT TSS_Error setMouseRelative (TSS_ID *id,* boolean *do_relative*)

Puts the emulated HID mouse in absolute or relative mode.

Please note that this change does not take effect immediately, and the sensor's settings must be committed and the sensor must be reset before the mouse will enter this mode.

Parameters

id	The identifier for the 3-Space device.
do_relative	Sets whether the mouse is to be relative or not. If the value is True, the mouse is relative. If the value is False, the mouse is in absolute mode.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.222 EXPORT TSS_Error setMultiRefChkVecAccelerometer (TSS_ID *id,* unsigned char *idx,* TSS_Vector *vec*)

Directly set accelerometer multi reference check vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the accelerometer to set the vector to.
vec	The vector to set at idx.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.223 EXPORT TSS_Error setMultiRefChkVecAccelerometerf3 (TSS_ID *id*, unsigned char *idx*, float * *vec*)

Directly set accelerometer multi reference check vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the accelerometer to set the vector to.
vec	The vector to set at idx as an array of three floats. The floats should be arranged: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.224 EXPORT TSS_Error setMultiRefChkVecCompass (TSS_ID *id*, unsigned char *idx*, TSS_Vector *vec*)

Directly set compass multi reference check vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the compass to set the vector to.
vec	The vector to set at idx.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.225 EXPORT TSS_Error setMultiRefChkVecCompassf3 (TSS_ID *id*, unsigned char *idx*, float * *vec*)

Directly set compass multi reference check vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the compass to set the vector to.
vec	The vector to set at idx as an array of three floats. The floats should be arranged: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.226 EXPORT TSS_Error setMultiRefResolution (TSS_ID *id,* unsigned char *num_division,* unsigned char *num_vecs*)

Sets number of cell divisions and number of nearby vectors per cell for the multi reference lookup table.

By default, the multiple reference vector mode only deals with orientations obtainable by successive rotations of 90 degrees about any of the three axes. This command can adjust it to accept orientations obtainable by 45 degree rotations. For 90 degrees, give a "number of cell divisions" of 4, and for 45 give 8. In addition, the number of vectors near to any given orientation which the scheme will check can be adjusted as well. If this value is 0, only the nearest orientation will be checked. The largest this value can be is 32. Also note that the larger this value is, the longer the multiple reference vector mode will take to run each cycle. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
num_division	The number of cell divisions.
num_vecs	The number of nearby vectors.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.227 EXPORT TSS_Error setMultiRefVecAccelerometer (TSS_ID *id,* unsigned char *idx,* TSS_Vector *vec*)

Directly set accelerometer multi reference vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the accelerometer to set the vector to.
vec	The vector to set at idx.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.228 EXPORT TSS_Error setMultiRefVecAccelerometerf3 (TSS_ID *id,* unsigned char *idx,* float * *vec*)

Directly set accelerometer multi reference vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the accelerometer to set the vector to.
vec	The vector to set at idx as an array of three floats. The floats should be arranged: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.229 EXPORT TSS_Error setMultiRefVecCompass (TSS_ID *id,* unsigned char *idx,* TSS_Vector *vec*)

Directly set compass multi reference vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the compass to set the vector to.
vec	The vector to set at idx.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.230 EXPORT TSS_Error setMultiRefVecCompassf3 (TSS_ID *id,* unsigned char *idx,* float * *vec*)

Directly set compass multi reference vector at the specified index to the specified vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the compass to set the vector to.
vec	The vector to set at idx as an array of three floats. The floats should be arranged: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.231 EXPORT TSS_Error setMultiRefVecZero (TSS_ID *id*)

Resets all reference vectors in the multi reference scheme to zero. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.232 EXPORT TSS_Error setMultiRefWeightPwr (TSS_ID *id,* float *pow*)

Set weighting power for multi reference vector weights. Multi reference vector weights are all raised to the weight power before they are summed and used in the calculation for the final reference vector. Setting this value nearer to 0 will cause the reference vectors to overlap more, and setting it nearer to infinity will cause the reference vectors to influence a smaller set of orientations. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
pow	The weight power to use.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.233 EXPORT TSS_Error setOrientationButton (TSS_ID *id,* char * *joy_or_mouse,* unsigned char *but_idx,* TSS_Vector *local_axis,* TSS_Vector *global_axis,* float *max_dist*)

Sets up a device's button such that it is 'pressed' when a reference vector aligns itself with a local vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
joy_or_mouse	A null-terminated string whose value may be either 'mouse' or 'joystick'.
but_idx	The value that defines which button on the emulated device to configure. This value is in the range 0 - 7.
local_axis	A normalized vector representing the sensor's local vector to track.
global_axis	A normalized vector representing the global vector to move the local vector towards for "pressing" (should not be colinear to the local vector).
max_dist	Defines how close the local vector's orientation must be to the global vector for the button to be 'pressed'.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.234 EXPORT TSS_Error setOversampleRate (TSS_ID *id,* unsigned char *rate*)

Sets oversample rate of data reads from the 3-Space Sensor's on-board sensors. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
rate	The amount of oversampling to perform. 1 turns off oversampling. 2+ sets the number of samples per frame.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.235 EXPORT TSS_Error setPanID (TSS_ID *id,* unsigned short *pan_id*)

Sets the Pan ID for the 3-Space Device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.

Parameters

id	The identifier for the 3-Space device.
pan_id	The Pan ID to set to the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.236 EXPORT TSS_Error setPhysicalButton (TSS_ID *id,* char * *joy_or_mouse,* unsigned char *but_idx,* char * *button_bind*)

Binds a sensor's physical button to an emulated device's button. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
joy_or_mouse	A null-terminated string whose value may be either 'mouse' or 'joystick'.
but_idx	The value that defines which button on the emulated device to configure. This value is in the range 0 - 7.
button_bind	A null-terminated string whose value may be 'left' or 'right'. This string defines which physical button to bind to the emulated device's button to as defined by but_idx. Designation of 'left' and 'right' assumes the sensor to be oriented such that the LED side of the sensor is facing up and the side of the sensor that contains the USB port is facing towards the user.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.237 EXPORT TSS_Error setRefVecAccelerometer (TSS_ID *id,* TSS_Vector *vec*)

Sets the static accelerometer reference vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
vec	The vector to set the accelerometer's reference vector to.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.238 EXPORT TSS_Error setRefVecAccelerometerf3 (TSS_ID *id,* float * *vec*)

Sets the static accelerometer reference vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
vec	The vector to set the accelerometer's reference vector to as an array of three floats. The floats should be arranged: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.239 EXPORT TSS_Error setRefVecCompass (TSS_ID *id,* TSS_Vector *vec*)

Sets the static compass reference vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
vec	The vector to set the compass's reference vector to.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.240 EXPORT TSS_Error setRefVecCompassf3 (TSS_ID *id,* float * *vec*)

Sets the static compass reference vector. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
vec	The vector to set the compass's reference vector to as an array of three floats. The floats should be arranged: x, y, z.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.241 EXPORT TSS_Error setRefVecCurrentOrient (TSS_ID *id*)

Uses the current tared orientation to set up the reference vector for the nearest orthogonal orientation. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.242 EXPORT TSS_Error setRefVecMode (TSS_ID *id,* unsigned char *mode*)

Sets reference vector mode to either 'single static', 'single auto', 'single auto continuous', or 'multi'.

Single static mode uses a certain reference vector for the compass and another certain reference vector for the accelerometer at all times.

Single auto mode uses (0,-1,0) as the reference vector for the accelerometer at all times and uses the current average angle between the accelerometer and compass to calculate the compass reference vector. After that this mode acts like single static mode.

Single auto continuous mode uses (0,-1,0) as the reference vector for the accelerometer at all times and uses the average angle between the accelerometer and compass to constantly redetermine the compass reference vector.

Multi mode uses a collection of reference vectors for the compass and accelerometer, and selects which ones to use before each step of the filter.

This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mode	The desired mode of operation. (0 for single static, 1 for single auto, 2 for single auto continuous, 3 for multi).

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.243 EXPORT TSS_Error setRunningAvgMode (TSS_ID *id,* unsigned char *mode*)

Selects the mode that the running-average method uses. 'Normal' uses a static running-average. 'Confidence' uses a running average that changes dynamically based upon the confidence factor. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mode	The mode to set the sensor's running average to. Possible values are: 0 - Normal mode 1 - Confidence Mode

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.244 EXPORT TSS_Error setScreenPointAxis (TSS_ID *id,* char * *joy_or_mouse,* char * *device_axis,* float *dist_from_screen,* float *dist_on_axis,* char * *collision_component,* char * *sensor_dir,* float *button_halt*)

Sets an axis of the desired emulated input device as a 'Screen Point Axis' style axis. This call will error if the inputted id is for a 3-Space Dongle.

An axis operating under this style projects a vector along the sensor's direction vector into a mathmatical plane. The collision point on the plane is then used to determine what the device's axis's current value is. The direction vector is rotated based on the orientation of the sensor.

Parameters

id	The identifier for the 3-Space device.
joy_or_mouse	A null-terminated string whose value may be either 'mouse' or 'joystick'. This string defines whether the device in question is a mouse or joystick.
device_axis	A null-terminated string whose value may be either 'X' or 'Y'. This string defines what axis of the device is to be configured.

dist_from_screen	A float whose value is the real world distance the sensor is from the user's screen. Must be the same units as dist_on_axis.
dist_on_axis	A float whose value is the real world length of the axis along the user's screen (width of screen for x-axis, height of screen for y-axis). Must be the same units as dist_from_screen.
collision_component	A null-terminated string whose value may be 'X', 'Y', or 'Z'. This string defines what component of the look vector's collision point on the virtual plane to use for manipulating the device's axis.
sensor_dir	A null-terminated string whose value may be 'X', 'Y', or 'Z'. This string defines which of the sensor's local axis to use for creating the vector to collide with the virtual plane.
button_halt	A float whose value is a pause time in milliseconds. When a button is pressed on the emulated device, transmission of changes to the axis is paused for the inputted amount of time to prevent undesired motion detection when pressing buttons.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.245 EXPORT TSS_Error setSerialNumber (TSS_ID *id,* unsigned int *serial*)

Sets the serial number for the 3-Space device in bootloader mode. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

Parameters

id	The identifier for the 3-Space device.
serial	The serial number to set to the device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.246 EXPORT TSS_Error setShakeButton (TSS_ID *id,* char * *joy_or_mouse,* unsigned char *but_idx,* float *threshold*)

Sets up an emulated device's button such that it is 'pressed' when the sensor is shaken. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
joy_or_mouse	A null-terminated string whose value may be either 'mouse' or 'joystick'.
but_idx	The value that defines which button on the emulated device to configure. This value is in the range 0 - 7.
threshold	Defines how many Gs of force must be experienced by the sensor before the button is 'pressed'.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.247 EXPORT TSS_Error setStaticRhoModeAccel (TSS_ID *id,* float *val*)

Sets the rho mode for the accelerometer to static, using the given value as rho. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
val	The value to use as the new rho value.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.248 EXPORT TSS_Error setStaticRhoModeCompass (TSS_ID *id,* float *val*)

Sets the rho mode for the compass to static, using the given value as rho. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
val	The value to use as the new rho value.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.249 EXPORT TSS_Error setTareCurrentOrient (TSS_ID *id*)

Sets current filtered orientation derived from its detected gravity and north vectors as the tare orientation. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.250 EXPORT TSS_Error setTareMatrix (TSS_ID *id,* TSS_Matrix *mat*)

Sets current filtered orientation to a passed in rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to set the tare orientation to.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.251 EXPORT TSS_Error setTareMatrixf9 (TSS_ID *id,* float * *mat*)

Sets current filtered orientation to a passed in rotation matrix. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
mat	The matrix to set the tare orientation to. The array of nine floats should be arranged in row-major order.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.252 EXPORT TSS_Error setTareQuaternion (TSS_ID *id,* TSS_Quaternion *quat*)

Sets current filtered orientation to a passed in quaternion. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
quat	The quaternion to set the tare orientation to.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.253 EXPORT TSS_Error setTareQuaternionf4 (TSS_ID *id,* float * *quat*)

Sets current filtered orientation to a passed in quaternion. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
quat	The quaternion to set the tare orientation to. The array of four floats should be structured as such: x, y, z, w.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.254 EXPORT TSS_Error setToFirmwareMode (TSS_ID *id*)

Resets the device to use firmware loaded to the 3-Space device. Will remain in bootloader mode if the firmware was corrupted. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.255 EXPORT TSS_Error setUARTRate (TSS_ID *id,* unsigned int *rate*)

Sets the baud rate of the physical UART. This call will error if the inputted id is not for a 3-Space USB Sensor or a 3-Space Embedded Sensor.

This setting does not need to be committed, but does not take effect until the sensor is reset. The baud rate will be set to the valid value nearest the requested value.

Parameters

id	The identifier for the 3-Space device.
rate	The desired baud rate of the UART.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.256 EXPORT TSS_Error setUpdateRate (TSS_ID *id,* unsigned int *rate*)

Sets the target update rate to the given rate. If the filter takes less time to update during a given frame than this rate specifies, the frame will be padded out to take the specified amount of time. If the filter takes more time to update than this rate, this rate will be ignored. This call will error if the inputted id is for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
rate	Represents the desired update rate in microseconds.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.257 EXPORT TSS_Error setUSBMode (TSS_ID *id,* unsigned char *mode*)

Sets the communication mode for USB.

All modes present a COM port with which to communicate with the USB device. FTDI and CDC each present a regular numbered port, whereas Unique presents a port named YEI_<serial number>. The sensor will change modes immediately.

Parameters

id	The identifier for the 3-Space device.
mode	The desired USB mode for the sensor. Valid values are: • 0 for CDC mode. • 1 for FTDI mode. • 2 for Unique mode.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.258 EXPORT TSS_Error setWirelessAddress (TSS_ID *id,* unsigned short *address*)

Sets the wireless address of the device. This call will error if the inputted id is not for a 3-Space Dongle or 3-Space Wireless Sensor.

Parameters

id	The identifier for the 3-Space device.
address	The wireless address to set the device to.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.259 EXPORT TSS_Error setWirelessHWID (TSS_ID *id,* unsigned char *idx,* unsigned int *hw_id*)

Sets the logical ID given by an index to the hardware ID (sensor serial number) given by hw_id. This call will error if the inputted id is not for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
idx	The index of the dongle's wireless table to read the hardware ID from.
hw_id	The serial number of the wireless sensor being written to the dongle's wireless table.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.260 EXPORT TSS_Error setWirelessRetries (TSS_ID *id,* unsigned char *retries*)

Set the number of times the dongle will retry a failed transmission to a wireless sensor. This call will error if the inputted id is not for a 3-Space Dongle.

Parameters

id	The identifier for the 3-Space device.
retries	The desired number of times to retry a transmission to a wireless sensor before giving up. The maximum value, as well as the default value, is 3.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.261 EXPORT TSS_Error setWriteStart (TSS_ID *id,* unsigned int *start_addr*)

Sets the memory address for writting firmware to the 3-Space device in bootloader mode. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

Parameters

id	The identifier for the 3-Space device.
start_addr	Denotes the start position for writting the firmware.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.262 EXPORT TSS_Error softwareReset (TSS_ID *id*)

Resets the device without powering it down.

Any settings saved to non-volatile memory will be restored and the sensor. During the reset, the sensor will be inactive for approximatly 5 seconds. Certain setting changes only take place when a reset has occurred.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.263 EXPORT TSS_Error startLogSession (TSS_ID *id*)

Begins a data logging session based on the contents of the capture.cfg file.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.264 EXPORT TSS_Error stopAsynchronous (TSS_ID *id*)

Stops all asynchronous transmissions from the wireless sensor. This call will error if the inputted id is not for a 3-Space Wireless Sensor.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.265 EXPORT TSS_Error stopLogSession (TSS_ID *id*)

Stops any active capture sessions. This call will error if the inputted id is not for a 3-Space Data-Logging Sensor.

Parameters

id	The identifier for the 3-Space device.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.266 EXPORT TSS_Error updateAsynchData ( )

Updates the internaly stored asynchronous data gathered from the connected 3-Space Dongles.

This function must be called in order to retrieve updated asynchronous data.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.267 EXPORT TSS_Error writeEncryptedData (TSS_ID *id,* unsigned char * *data,* unsigned int *data_size*)

Writes encrypted data to the 3-Space Sensor. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

Parameters

id	The identifier for the 3-Space device.
data	The array of bytes to write to the 3-Space device.
data_size	The number of bytes in the array.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

4.1.4.268 EXPORT TSS_Error writeUnencryptedData (TSS_ID *id,* unsigned char * *data,* unsigned int *data_size*)

Writes unencrypted data to the 3-Space Sensor. This call will error if the inputted id is not for a 3-Space device in Bootloader mode.

Parameters

id	The identifier for the 3-Space device.
data	The array of bytes to write to the 3-Space device.
data_size	The number of bytes in the array.

Returns

An error code indicating either success or failure to execute the call. The code will also indicate the reason for the failure.

Index

addSensorToDongle
    yei_threespace_api.h, 32
asynchGetAccelerometerNormalized
    yei_threespace_api.h, 33
asynchGetAccelerometerRaw
    yei_threespace_api.h, 33
asynchGetAccelerometerUnfiltered
    yei_threespace_api.h, 34
asynchGetAllSensorsNormalized
    yei_threespace_api.h, 34
asynchGetAllSensorsRaw
    yei_threespace_api.h, 35
asynchGetButtonState
    yei_threespace_api.h, 35
asynchGetCompassNormalized
    yei_threespace_api.h, 36
asynchGetCompassRaw
    yei_threespace_api.h, 36
asynchGetCompassUnfiltered
    yei_threespace_api.h, 37
asynchGetFiltNorthEarthVecSensFrame
    yei_threespace_api.h, 37
asynchGetFiltOrientAxisAngle
    yei_threespace_api.h, 38
asynchGetFiltOrientEuler
    yei_threespace_api.h, 38
asynchGetFiltOrientFwdDwn
    yei_threespace_api.h, 39
asynchGetFiltOrientMat
    yei_threespace_api.h, 39
asynchGetFiltOrientQuat
    yei_threespace_api.h, 40
asynchGetFiltTaredFwdDwnVecSensFrame
    yei_threespace_api.h, 40
asynchGetFiltTaredOrientAxisAngle
    yei_threespace_api.h, 41
asynchGetFiltTaredOrientEuler
    yei_threespace_api.h, 41
asynchGetFiltTaredOrientFwdDwn
    yei_threespace_api.h, 42
asynchGetFiltTaredOrientMatrix
    yei_threespace_api.h, 42
asynchGetFiltTaredOrientQuat
    yei_threespace_api.h, 43
asynchGetGyroNormalized
    yei_threespace_api.h, 43
asynchGetGyroRaw
    yei_threespace_api.h, 44
Axis_Order
    yei_threespace_api.h, 30, 31
AxisDirections, 5

BootloaderData, 5
ButtonState, 6
    yei_threespace_api.h, 30

calibrateGyro
    yei_threespace_api.h, 44
closeTSDevice
    yei_threespace_api.h, 44
ComInfo, 6
ComPort, 7
commitSettings
    yei_threespace_api.h, 44
commitWirelessSettings
    yei_threespace_api.h, 45
createTSDevice
    yei_threespace_api.h, 45
createTSDeviceStr
    yei_threespace_api.h, 45

day
    TSS_Clock, 9
disableAxis
    yei_threespace_api.h, 46
disableButton
    yei_threespace_api.h, 46
disableMassStorageMode
    yei_threespace_api.h, 46
disableWatchdogTimer
    yei_threespace_api.h, 46

enableMassStorageMode
    yei_threespace_api.h, 47
enableWatchdogTimer
    yei_threespace_api.h, 47
enterFirmwareUpdateMode
    yei_threespace_api.h, 47

finishWrite
    yei_threespace_api.h, 48
formatInitSDCard
    yei_threespace_api.h, 48

getAccelerometerCalibrationParam
    yei_threespace_api.h, 48
getAccelerometerCalibrationParamf9f3
    yei_threespace_api.h, 48
getAccelerometerEnabled
    yei_threespace_api.h, 49

getAccelerometerNormalized
yei_threespace_api.h, 49
getAccelerometerNormalizedf3
yei_threespace_api.h, 49
getAccelerometerRange
yei_threespace_api.h, 50
getAccelerometerRaw
yei_threespace_api.h, 50
getAccelerometerRawf3
yei_threespace_api.h, 50
getAccelerometerUnfiltered
yei_threespace_api.h, 51
getAccelerometerUnfilteredf3
yei_threespace_api.h, 51
getActualUpdateRate
yei_threespace_api.h, 51
getAllChannelNoise
yei_threespace_api.h, 51
getAllSensorsNormalized
yei_threespace_api.h, 52
getAllSensorsNormalizedf9
yei_threespace_api.h, 52
getAllSensorsRaw
yei_threespace_api.h, 52
getAllSensorsRawf9
yei_threespace_api.h, 53
getAsynchDataFloat
yei_threespace_api.h, 53
getAsynchDataUChar
yei_threespace_api.h, 53
getAsynchDataUInt
yei_threespace_api.h, 54
getAsynchDataUShort
yei_threespace_api.h, 54
getAvailableComPorts
yei_threespace_api.h, 54
getAvgPercent
yei_threespace_api.h, 55
getAxisDirections
yei_threespace_api.h, 55
getBatteryLife
yei_threespace_api.h, 56
getBatteryStatus
yei_threespace_api.h, 56
getBatteryVoltage
yei_threespace_api.h, 56
getButtonGyroDisableLength
yei_threespace_api.h, 56
getButtonState
yei_threespace_api.h, 57
getChannel
yei_threespace_api.h, 57
getClock
yei_threespace_api.h, 57
getClockSpeed
yei_threespace_api.h, 58
getClockc6
yei_threespace_api.h, 58
getComPortInfo
yei_threespace_api.h, 61
getCompassCalibrationParam
yei_threespace_api.h, 58
getCompassCalibrationParamf9f3
yei_threespace_api.h, 59
getCompassEnabled
yei_threespace_api.h, 59
getCompassNormalized
yei_threespace_api.h, 59
getCompassNormalizedf3
yei_threespace_api.h, 59
getCompassRange
yei_threespace_api.h, 60
getCompassRaw
yei_threespace_api.h, 60
getCompassRawf3
yei_threespace_api.h, 60
getCompassUnfiltered
yei_threespace_api.h, 61
getCompassUnfilteredf3
yei_threespace_api.h, 61
getConfidence
yei_threespace_api.h, 62
getDesiredUpdateRate
yei_threespace_api.h, 62
getDeviceType
yei_threespace_api.h, 62
getFiltGyroRate
yei_threespace_api.h, 63
getFiltGyroRatef4
yei_threespace_api.h, 63
getFiltNorthEarthVecSensFrame
yei_threespace_api.h, 64
getFiltNorthEarthVecSensFramef3f3
yei_threespace_api.h, 64
getFiltOrientAxisAngle
yei_threespace_api.h, 64
getFiltOrientAxisAnglef4
yei_threespace_api.h, 65
getFiltOrientEuler
yei_threespace_api.h, 65
getFiltOrientEulerf3
yei_threespace_api.h, 65
getFiltOrientFwdDwn
yei_threespace_api.h, 66
getFiltOrientFwdDwnf3f3
yei_threespace_api.h, 66
getFiltOrientMat
yei_threespace_api.h, 66
getFiltOrientMatf9
yei_threespace_api.h, 66
getFiltOrientQuat
yei_threespace_api.h, 67
getFiltOrientQuatf4
yei_threespace_api.h, 67
getFiltTaredFwdDwnVecSensFrame
yei_threespace_api.h, 67

getFiltTaredFwdDwnVecSensFramef3f3
yei_threespace_api.h, 68
getFiltTaredOrientAxisAngle
yei_threespace_api.h, 68
getFiltTaredOrientAxisAnglef4
yei_threespace_api.h, 68
getFiltTaredOrientEuler
yei_threespace_api.h, 69
getFiltTaredOrientEulerf3
yei_threespace_api.h, 69
getFiltTaredOrientFwdDwn
yei_threespace_api.h, 69
getFiltTaredOrientFwdDwnf3f3
yei_threespace_api.h, 69
getFiltTaredOrientMat
yei_threespace_api.h, 70
getFiltTaredOrientMatf9
yei_threespace_api.h, 70
getFiltTaredOrientQuat
yei_threespace_api.h, 70
getFiltTaredOrientQuatf4
yei_threespace_api.h, 71
getFilterMode
yei_threespace_api.h, 63
getFirstAvailableTSSComPort
yei_threespace_api.h, 71
getFirstAvailableUnknownComPort
yei_threespace_api.h, 71
getFriendlyName
yei_threespace_api.h, 71
getGyroCalibrationParam
yei_threespace_api.h, 72
getGyroCalibrationParamf9f3
yei_threespace_api.h, 72
getGyroEnabled
yei_threespace_api.h, 72
getGyroNormalized
yei_threespace_api.h, 73
getGyroNormalizedf3
yei_threespace_api.h, 73
getGyroRange
yei_threespace_api.h, 73
getGyroRaw
yei_threespace_api.h, 74
getGyroRawf3
yei_threespace_api.h, 74
getHIDUpdateRate
yei_threespace_api.h, 74
getHardwareVersion
yei_threespace_api.h, 74
getInformation
yei_threespace_api.h, 75
getInterruptStatus
yei_threespace_api.h, 75
getInterruptType
yei_threespace_api.h, 75
getJoystickEnabled
yei_threespace_api.h, 76
getJoystickLogicalID
yei_threespace_api.h, 76
getJoystickMousePresent
yei_threespace_api.h, 77
getKalmanMat
yei_threespace_api.h, 77
getKalmanMatf9
yei_threespace_api.h, 77
getLEDColor
yei_threespace_api.h, 77
getLEDColorf3
yei_threespace_api.h, 78
getLEDMode
yei_threespace_api.h, 78
getLookupTblVertVal
yei_threespace_api.h, 78
getLookupTblVertValf3
yei_threespace_api.h, 79
getMouseEnabled
yei_threespace_api.h, 79
getMouseLogicalID
yei_threespace_api.h, 80
getMouseRelative
yei_threespace_api.h, 80
getMultiRefChkVecAccelerometer
yei_threespace_api.h, 80
getMultiRefChkVecAccelerometerf3
yei_threespace_api.h, 81
getMultiRefChkVecCompass
yei_threespace_api.h, 81
getMultiRefChkVecCompassf3
yei_threespace_api.h, 81
getMultiRefResolution
yei_threespace_api.h, 81
getMultiRefVecAccelerometer
yei_threespace_api.h, 82
getMultiRefVecAccelerometerf3
yei_threespace_api.h, 82
getMultiRefVecCompass
yei_threespace_api.h, 82
getMultiRefVecCompassf3
yei_threespace_api.h, 83
getMultiRefWeightPwr
yei_threespace_api.h, 83
getNextAvailableTSSComPort
yei_threespace_api.h, 84
getNextAvailableUnknownComPort
yei_threespace_api.h, 84
getNumMultiRefCells
yei_threespace_api.h, 84
getOversampleRate
yei_threespace_api.h, 84
getPanID
yei_threespace_api.h, 85
getReceptionStrength
yei_threespace_api.h, 85
getRefVecAccelerometer
yei_threespace_api.h, 85

getRefVecAccelerometerf3
yei_threespace_api.h, 86
getRefVecCompass
yei_threespace_api.h, 86
getRefVecCompassf3
yei_threespace_api.h, 86
getRefVecMode
yei_threespace_api.h, 86
getRhoDataAccelerometer
yei_threespace_api.h, 87
getRhoDataCompass
yei_threespace_api.h, 87
getRunningAvgMode
yei_threespace_api.h, 88
getSensorFromDongle
yei_threespace_api.h, 88
getSerialNumber
yei_threespace_api.h, 88
getSerialNumberHex
yei_threespace_api.h, 88
getSoftwareVersion
yei_threespace_api.h, 89
getTareOrientMat
yei_threespace_api.h, 89
getTareOrientMatf9
yei_threespace_api.h, 89
getTareOrientQuat
yei_threespace_api.h, 90
getTareOrientQuatf4
yei_threespace_api.h, 90
getTemperatureCelsius
yei_threespace_api.h, 90
getTemperatureFahrenheit
yei_threespace_api.h, 90
getUARTRate
yei_threespace_api.h, 91
getUSBMode
yei_threespace_api.h, 91
getWirelessAddress
yei_threespace_api.h, 91
getWirelessHWID
yei_threespace_api.h, 92
getWirelessRetries
yei_threespace_api.h, 92
getWirelessSlotsOpen
yei_threespace_api.h, 92

hour
TSS_Clock, 9

include/yei_threespace_api.h, 13
isConnected
yei_threespace_api.h, 93
isFirmwareValid
yei_threespace_api.h, 93

minute
TSS_Clock, 9
month
TSS_Clock, 9

resetKalmanFilter
yei_threespace_api.h, 93
restoreFactorySettings
yei_threespace_api.h, 93

second
TSS_Clock, 10
SensorData, 7
yei_threespace_api.h, 30
setAccelerometerCalibrationParam
yei_threespace_api.h, 94
setAccelerometerCalibrationParamf9f3
yei_threespace_api.h, 94
setAccelerometerEnabled
yei_threespace_api.h, 94
setAccelerometerRange
yei_threespace_api.h, 95
setAvgPercent
yei_threespace_api.h, 95
setAxisDirections
yei_threespace_api.h, 95
setButtonGyroDisableLength
yei_threespace_api.h, 96
setChannel
yei_threespace_api.h, 96
setClock
yei_threespace_api.h, 96
setClockSpeed
yei_threespace_api.h, 97
setClockc6
yei_threespace_api.h, 97
setCompassCalibrationParam
yei_threespace_api.h, 97
setCompassCalibrationParamf9f3
yei_threespace_api.h, 98
setCompassEnabled
yei_threespace_api.h, 98
setCompassRange
yei_threespace_api.h, 98
setConfidenceRhoModeAccel
yei_threespace_api.h, 99
setConfidenceRhoModeCompass
yei_threespace_api.h, 99
setFilterMode
yei_threespace_api.h, 100
setGlobalAxis
yei_threespace_api.h, 100
setGyroCalibrationParam
yei_threespace_api.h, 100
setGyroCalibrationParamf9f3
yei_threespace_api.h, 101
setGyroEnabled
yei_threespace_api.h, 101
setGyroRange
yei_threespace_api.h, 102
setHIDUpdateRate
yei_threespace_api.h, 102

setInterruptType
yei_threespace_api.h, 102
setJoystickEnabled
yei_threespace_api.h, 103
setJoystickLogicalID
yei_threespace_api.h, 103
setJoystickMousePresent
yei_threespace_api.h, 103
setLEDColor
yei_threespace_api.h, 104
setLEDColorf3
yei_threespace_api.h, 104
setLEDMode
yei_threespace_api.h, 104
setLookupTblVertVal
yei_threespace_api.h, 105
setLookupTblVertValf3
yei_threespace_api.h, 105
setMouseEnabled
yei_threespace_api.h, 105
setMouseLogicalID
yei_threespace_api.h, 106
setMouseRelative
yei_threespace_api.h, 106
setMultiRefChkVecAccelerometer
yei_threespace_api.h, 106
setMultiRefChkVecAccelerometerf3
yei_threespace_api.h, 107
setMultiRefChkVecCompass
yei_threespace_api.h, 107
setMultiRefChkVecCompassf3
yei_threespace_api.h, 107
setMultiRefResolution
yei_threespace_api.h, 107
setMultiRefVecAccelerometer
yei_threespace_api.h, 108
setMultiRefVecAccelerometerf3
yei_threespace_api.h, 108
setMultiRefVecCompass
yei_threespace_api.h, 108
setMultiRefVecCompassf3
yei_threespace_api.h, 109
setMultiRefVecZero
yei_threespace_api.h, 109
setMultiRefWeightPwr
yei_threespace_api.h, 109
setOrientationButton
yei_threespace_api.h, 110
setOversampleRate
yei_threespace_api.h, 110
setPanID
yei_threespace_api.h, 110
setPhysicalButton
yei_threespace_api.h, 111
setRefVecAccelerometer
yei_threespace_api.h, 111
setRefVecAccelerometerf3
yei_threespace_api.h, 111
setRefVecCompass
yei_threespace_api.h, 112
setRefVecCompassf3
yei_threespace_api.h, 112
setRefVecCurrentOrient
yei_threespace_api.h, 112
setRefVecMode
yei_threespace_api.h, 112
setRunningAvgMode
yei_threespace_api.h, 113
setScreenPointAxis
yei_threespace_api.h, 113
setSerialNumber
yei_threespace_api.h, 114
setShakeButton
yei_threespace_api.h, 114
setStaticRhoModeAccel
yei_threespace_api.h, 114
setStaticRhoModeCompass
yei_threespace_api.h, 115
setTareCurrentOrient
yei_threespace_api.h, 115
setTareMatrix
yei_threespace_api.h, 115
setTareMatrixf9
yei_threespace_api.h, 116
setTareQuaternion
yei_threespace_api.h, 116
setTareQuaternionf4
yei_threespace_api.h, 116
setToFirmwareMode
yei_threespace_api.h, 116
setUARTRate
yei_threespace_api.h, 117
setUSBMode
yei_threespace_api.h, 117
setUpdateRate
yei_threespace_api.h, 117
setWirelessAddress
yei_threespace_api.h, 118
setWirelessHWID
yei_threespace_api.h, 118
setWirelessRetries
yei_threespace_api.h, 118
setWriteStart
yei_threespace_api.h, 119
softwareReset
yei_threespace_api.h, 119
startLogSession
yei_threespace_api.h, 119
stopAsynchronous
yei_threespace_api.h, 119
stopLogSession
yei_threespace_api.h, 120

TSS_ERROR_NO_SENSOR_FOUND
yei_threespace_api.h, 32
TSS_ERROR_OLD_FIRMWARE
yei_threespace_api.h, 32

TSS_ERROR_READ
yei_threespace_api.h, 32
TSS_ERROR_USB_CONNECTION
yei_threespace_api.h, 32
TSS_ERROR_WIR_CONNECTION
yei_threespace_api.h, 32
TSS_ERROR_WRITE
yei_threespace_api.h, 32
TSS_INVALID_COMMAND
yei_threespace_api.h, 32
TSS_INVALID_ID
yei_threespace_api.h, 32
TSS_INVALID_IDX
yei_threespace_api.h, 32
TSS_NO_ERROR
yei_threespace_api.h, 32
TSS_WARNING_OLD_FIRMWARE
yei_threespace_api.h, 32
TSS_AxisAngle, 8
yei_threespace_api.h, 30
TSS_Clock, 9
day, 9
hour, 9
minute, 9
month, 9
second, 10
year, 10
TSS_Color, 10
TSS_Error
yei_threespace_api.h, 32
TSS_Euler, 10
yei_threespace_api.h, 31
TSS_Matrix, 11
yei_threespace_api.h, 31
TSS_Quaternion, 12
TSS_Vector, 12

updateAsynchData
yei_threespace_api.h, 120

writeEncryptedData
yei_threespace_api.h, 120
writeUnencryptedData
yei_threespace_api.h, 120

XYZ
yei_threespace_api.h, 31
XZY
yei_threespace_api.h, 31

YXZ
yei_threespace_api.h, 31
YZX
yei_threespace_api.h, 31
year
TSS_Clock, 10
yei_threespace_api.h
TSS_ERROR_NO_SENSOR_FOUND, 32
TSS_ERROR_OLD_FIRMWARE, 32
TSS_ERROR_READ, 32
TSS_ERROR_USB_CONNECTION, 32
TSS_ERROR_WIR_CONNECTION, 32
TSS_ERROR_WRITE, 32
TSS_INVALID_COMMAND, 32
TSS_INVALID_ID, 32
TSS_INVALID_IDX, 32
TSS_NO_ERROR, 32
TSS_WARNING_OLD_FIRMWARE, 32
XYZ, 31
XZY, 31
YXZ, 31
YZX, 31
ZXY, 31
ZYX, 31
yei_threespace_api.h
addSensorToDongle, 32
asynchGetAccelerometerNormalized, 33
asynchGetAccelerometerRaw, 33
asynchGetAccelerometerUnfiltered, 34
asynchGetAllSensorsNormalized, 34
asynchGetAllSensorsRaw, 35
asynchGetButtonState, 35
asynchGetCompassNormalized, 36
asynchGetCompassRaw, 36
asynchGetCompassUnfiltered, 37
asynchGetFiltNorthEarthVecSensFrame, 37
asynchGetFiltOrientAxisAngle, 38
asynchGetFiltOrientEuler, 38
asynchGetFiltOrientFwdDwn, 39
asynchGetFiltOrientMat, 39
asynchGetFiltOrientQuat, 40
asynchGetFiltTaredFwdDwnVecSensFrame, 40
asynchGetFiltTaredOrientAxisAngle, 41
asynchGetFiltTaredOrientEuler, 41
asynchGetFiltTaredOrientFwdDwn, 42
asynchGetFiltTaredOrientMatrix, 42
asynchGetFiltTaredOrientQuat, 43
asynchGetGyroNormalized, 43
asynchGetGyroRaw, 44
Axis_Order, 30, 31
ButtonState, 30
calibrateGyro, 44
closeTSDevice, 44
commitSettings, 44
commitWirelessSettings, 45
createTSDevice, 45
createTSDeviceStr, 45
disableAxis, 46
disableButton, 46
disableMassStorageMode, 46
disableWatchdogTimer, 46
enableMassStorageMode, 47
enableWatchdogTimer, 47
enterFirmwareUpdateMode, 47
finishWrite, 48
formatInitSDCard, 48
getAccelerometerCalibrationParam, 48

getAccelerometerCalibrationParamf9f3, 48
getAccelerometerEnabled, 49
getAccelerometerNormalized, 49
getAccelerometerNormalizedf3, 49
getAccelerometerRange, 50
getAccelerometerRaw, 50
getAccelerometerRawf3, 50
getAccelerometerUnfiltered, 51
getAccelerometerUnfilteredf3, 51
getActualUpdateRate, 51
getAllChannelNoise, 51
getAllSensorsNormalized, 52
getAllSensorsNormalizedf9, 52
getAllSensorsRaw, 52
getAllSensorsRawf9, 53
getAsynchDataFloat, 53
getAsynchDataUChar, 53
getAsynchDataUInt, 54
getAsynchDataUShort, 54
getAvailableComPorts, 54
getAvgPercent, 55
getAxisDirections, 55
getBatteryLife, 56
getBatteryStatus, 56
getBatteryVoltage, 56
getButtonGyroDisableLength, 56
getButtonState, 57
getChannel, 57
getClock, 57
getClockSpeed, 58
getClockc6, 58
getComPortInfo, 61
getCompassCalibrationParam, 58
getCompassCalibrationParamf9f3, 59
getCompassEnabled, 59
getCompassNormalized, 59
getCompassNormalizedf3, 59
getCompassRange, 60
getCompassRaw, 60
getCompassRawf3, 60
getCompassUnfiltered, 61
getCompassUnfilteredf3, 61
getConfidence, 62
getDesiredUpdateRate, 62
getDeviceType, 62
getFiltGyroRate, 63
getFiltGyroRatef4, 63
getFiltNorthEarthVecSensFrame, 64
getFiltNorthEarthVecSensFramef3f3, 64
getFiltOrientAxisAngle, 64
getFiltOrientAxisAnglef4, 65
getFiltOrientEuler, 65
getFiltOrientEulerf3, 65
getFiltOrientFwdDwn, 66
getFiltOrientFwdDwnf3f3, 66
getFiltOrientMat, 66
getFiltOrientMatf9, 66
getFiltOrientQuat, 67
getFiltOrientQuatf4, 67
getFiltTaredFwdDwnVecSensFrame, 67
getFiltTaredFwdDwnVecSensFramef3f3, 68
getFiltTaredOrientAxisAngle, 68
getFiltTaredOrientAxisAnglef4, 68
getFiltTaredOrientEuler, 69
getFiltTaredOrientEulerf3, 69
getFiltTaredOrientFwdDwn, 69
getFiltTaredOrientFwdDwnf3f3, 69
getFiltTaredOrientMat, 70
getFiltTaredOrientMatf9, 70
getFiltTaredOrientQuat, 70
getFiltTaredOrientQuatf4, 71
getFilterMode, 63
getFirstAvailableTSSComPort, 71
getFirstAvailableUnknownComPort, 71
getFriendlyName, 71
getGyroCalibrationParam, 72
getGyroCalibrationParamf9f3, 72
getGyroEnabled, 72
getGyroNormalized, 73
getGyroNormalizedf3, 73
getGyroRange, 73
getGyroRaw, 74
getGyroRawf3, 74
getHIDUpdateRate, 74
getHardwareVersion, 74
getInformation, 75
getInterruptStatus, 75
getInterruptType, 75
getJoystickEnabled, 76
getJoystickLogicalID, 76
getJoystickMousePresent, 77
getKalmanMat, 77
getKalmanMatf9, 77
getLEDColor, 77
getLEDColorf3, 78
getLEDMode, 78
getLookupTblVertVal, 78
getLookupTblVertValf3, 79
getMouseEnabled, 79
getMouseLogicalID, 80
getMouseRelative, 80
getMultiRefChkVecAccelerometer, 80
getMultiRefChkVecAccelerometerf3, 81
getMultiRefChkVecCompass, 81
getMultiRefChkVecCompassf3, 81
getMultiRefResolution, 81
getMultiRefVecAccelerometer, 82
getMultiRefVecAccelerometerf3, 82
getMultiRefVecCompass, 82
getMultiRefVecCompassf3, 83
getMultiRefWeightPwr, 83
getNextAvailableTSSComPort, 84
getNextAvailableUnknownComPort, 84
getNumMultiRefCells, 84
getOversampleRate, 84
getPanID, 85

getReceptionStrength, 85
getRefVecAccelerometer, 85
getRefVecAccelerometerf3, 86
getRefVecCompass, 86
getRefVecCompassf3, 86
getRefVecMode, 86
getRhoDataAccelerometer, 87
getRhoDataCompass, 87
getRunningAvgMode, 88
getSensorFromDongle, 88
getSerialNumber, 88
getSerialNumberHex, 88
getSoftwareVersion, 89
getTareOrientMat, 89
getTareOrientMatf9, 89
getTareOrientQuat, 90
getTareOrientQuatf4, 90
getTemperatureCelsius, 90
getTemperatureFahrenheit, 90
getUARTRate, 91
getUSBMode, 91
getWirelessAddress, 91
getWirelessHWID, 92
getWirelessRetries, 92
getWirelessSlotsOpen, 92
isConnected, 93
isFirmwareValid, 93
resetKalmanFilter, 93
restoreFactorySettings, 93
SensorData, 30
setAccelerometerCalibrationParam, 94
setAccelerometerCalibrationParamf9f3, 94
setAccelerometerEnabled, 94
setAccelerometerRange, 95
setAvgPercent, 95
setAxisDirections, 95
setButtonGyroDisableLength, 96
setChannel, 96
setClock, 96
setClockSpeed, 97
setClockc6, 97
setCompassCalibrationParam, 97
setCompassCalibrationParamf9f3, 98
setCompassEnabled, 98
setCompassRange, 98
setConfidenceRhoModeAccel, 99
setConfidenceRhoModeCompass, 99
setFilterMode, 100
setGlobalAxis, 100
setGyroCalibrationParam, 100
setGyroCalibrationParamf9f3, 101
setGyroEnabled, 101
setGyroRange, 102
setHIDUpdateRate, 102
setInterruptType, 102
setJoystickEnabled, 103
setJoystickLogicalID, 103
setJoystickMousePresent, 103
setLEDColor, 104
setLEDColorf3, 104
setLEDMode, 104
setLookupTblVertVal, 105
setLookupTblVertValf3, 105
setMouseEnabled, 105
setMouseLogicalID, 106
setMouseRelative, 106
setMultiRefChkVecAccelerometer, 106
setMultiRefChkVecAccelerometerf3, 107
setMultiRefChkVecCompass, 107
setMultiRefChkVecCompassf3, 107
setMultiRefResolution, 107
setMultiRefVecAccelerometer, 108
setMultiRefVecAccelerometerf3, 108
setMultiRefVecCompass, 108
setMultiRefVecCompassf3, 109
setMultiRefVecZero, 109
setMultiRefWeightPwr, 109
setOrientationButton, 110
setOversampleRate, 110
setPanID, 110
setPhysicalButton, 111
setRefVecAccelerometer, 111
setRefVecAccelerometerf3, 111
setRefVecCompass, 112
setRefVecCompassf3, 112
setRefVecCurrentOrient, 112
setRefVecMode, 112
setRunningAvgMode, 113
setScreenPointAxis, 113
setSerialNumber, 114
setShakeButton, 114
setStaticRhoModeAccel, 114
setStaticRhoModeCompass, 115
setTareCurrentOrient, 115
setTareMatrix, 115
setTareMatrixf9, 116
setTareQuaternion, 116
setTareQuaternionf4, 116
setToFirmwareMode, 116
setUARTRate, 117
setUSBMode, 117
setUpdateRate, 117
setWirelessAddress, 118
setWirelessHWID, 118
setWirelessRetries, 118
setWriteStart, 119
softwareReset, 119
startLogSession, 119
stopAsynchronous, 119
stopLogSession, 120
TSS_AxisAngle, 30
TSS_Error, 32
TSS_Euler, 31
TSS_Matrix, 31
updateAsynchData, 120
writeEncryptedData, 120

writeUnencryptedData, 120

ZXY
yei_threespace_api.h, 31
ZYX
yei_threespace_api.h, 31